

Design and Development of a Mobile Multi-Modal Data Acquisition Platform for Social Robot Navigation Benchmarking

Mohammad Foroutan Moharrami
Student ID: 240270344

Supervisor(s): Dr. Luis Manso,
Dr. Zhengjia Xu,
Dr. James Pickering

Department of Mechanical, Biomedical and Design Engineering
College of Engineering and Physical Sciences

April 2026

Abstract

This project focuses on designing and implementing a modular, multi-modal data acquisition platform using the Clearpath Jackal mobile robot. The system was created to reliably and synchronously collect robot sensor data and visual data from multiple cameras in real-world environments, which can support for further research in social mobile robotics and perception.

The platform was rebuilt from a non-working state and redeveloped using Ubuntu 22.04 and ROS 2 Humble. This provides a solid base for modern robotic applications. The main functions include SLAM-based mapping, AMCL localization, and Nav2-based autonomous navigation. We created a structured ROS 2 workspace and configuration framework to ensure it is modular and easy to maintain.

A significant contribution of this work is the development of a custom orchestration layer. This layer coordinates navigation and data recording processes. It allows various operational modes, such as autonomous navigation, teleoperation-based recording, and passive data collection, which enable capturing a range of scenarios.

Alongside this, we implemented a distributed multi-camera system using Jetson-based hardware and NVIDIA DeepStream. The setup integrates two USB cameras and six RTSP streams into a synchronized pipeline. It produces tiled video recordings along with frame-level timestamp logs. The synchronization between the robot and visual data was achieved through SSH-triggered recording and timestamp alignment.

We faced challenges with system performance, especially high CPU usage during localization and navigation, which was tackled by managing LiDAR data queues and optimizing parameters, leading to stable real-time operation.

The resulting platform showcases a strong, flexible, and scalable solution for multi-modal data acquisition in mobile robotics. It lays the groundwork for future work in structured data processing, perception, and learning-based navigation.

Keywords: ROS 2, Multi-Camera Systems, LiDAR-Based Navigation, DeepStream, Data Synchronisation, Multi-Modal Data Acquisition

Table of Contents

Chapter 1 – Introduction	6
1.1 Background	6
1.2 Problem Statement	6
1.3 Aim and Objectives	7
1.4 Key Achievements & Technical Contributions	8
1.5 Project Scope and Limitations	8
1.6 Dissertation Structure	8
Chapter 2 – Literature Review	9
2.1 Social Robot Navigation (SocNav)	9
2.2 Evaluation Metrics in Social Navigation	10
2.3 Datasets for Social Navigation	11
2.4 Robotic Middleware and Navigation Framework	11
2.4.1 ROS and ROS 2	11
2.4.2 Nav2 Navigation Stack	11
2.5 Data Acquisition Challenges in Robotics	12
2.6 Research Gap	13
2.7 Summary	13
Chapter 3 – System Overview	14
3.1 System Overview	14
3.2 Hardware Architecture	14
3.2.1 Robotic Platform	14
3.2.2 Multi-Camera Acquisition System	15
3.3 Software Architecture	17
3.4 System Summary	18
Chapter 4 – System Implementation	18
4.1 System Reconstruction	18
4.1.1 Clearpath OS Deployment	18
4.1.2 Initial Configuration	19
4.1.3 Internal Communication	19
4.1.4 External Connectivity	20
4.1.5 Connectivity Validation	21
4.2 Workspace and Package Architecture	21
4.3 Configuration Architecture	24
4.3.1 SLAM Configuration	24
4.3.2 Localisation (AMCL)	25
4.3.3 Navigation (Nav2)	26
4.3.4 LiDAR Integration	28
4.3.5 Robot-Level Configuration	28
4.4 Node Implementation and Launch System	29

4.4.1 Orchestration Node	29
4.4.2 Launch File Configuration	31
4.5 SSH Communication and Remote Access	32
4.6 System Optimisation	33
4.7 Camera System Implementation	34
4.7.1 Architecture	34
4.7.2 Camera Input	35
4.7.3 Data Flow	36
4.7.4 Synchronisation	36
4.7.5 Visual Composition	36
4.7.6 Encoding & Output	37
4.7.7 Remote Units	37
4.7.8 Power & Wiring	38
4.8 Summary	40
Chapter 5 – System Results and Outputs	40
5.1 Dataset Overview	40
5.2 Mapping Results	41
5.3 Localisation & Navigation Performance	41
5.4 Multi-Modal Recording Output	43
5.4.1 ROS Bag Data	43
5.4.2 Camera Output	45
5.4.3 Synchronisation	45
5.4.4 Temporal Performance	46
5.5 System Performance	47
5.6 Summary	48
Chapter 6 – Discussion	48
6.1 System Strength	49
6.2 Engineering Challenges	49
6.3 System Limitations	50
6.4 Comparison with Existing Approaches	51
6.5 Future Work	51
Chapter 7 – Conclusion	52
References	53
Appendix	56

List of Figures

Figure 3.1: High-Level System Architecture	14
Figure 3.2.1: Clearpath Jackal Robotic Platform	15
Figure 3.2.2-a: Camera Array Design	15
Figure 3.2.2-b: Camera System Architecture	16
Figure 3.2.2-c: Final Robot Setup	16
Figure 4.5: System Communication via SSH	33
Figure 4.7.3: Camera System Data Flow	35
Figure 4.7.8-a: UPP Battery	38
Figure 4.7.8-b: Mean Well Converter	38
Figure 4.7.8-c: Power Wiring Diagram	39
Figure 5.2: Mapping Results (MB-306 & LG-39)	41
Figure 5.3-a: RViz Navigation Screenshot	42
Figure 5.3-b: Indoor & Outdoor Navigation	43
Figure 5.4.1: ROS Bag Sample	44
Figure 5.4.2: Multi-Camera Output	45
Figure 5.4.3: Time Synchronisation Comparison	46
Figure 5.4.4: Timestamp JSON Sample	47
Figure 5.5: CPU Performance Before/After Optimisation	48

List of Tables

Table 3.3: Key ROS Topics and Definitions	17
Table 4.2: Directory Structure Description	24
Table 4.4.2: Launch Components and Responsibilities	32
Table 4.7: Multi-Camera System Components	39
Table 5.1: Dataset Summary	40
Table 5.4.1: Topic Frequency Analysis	43
Table 5.4.4: Temporal Performance Metrics	46

1 Introduction

1.1 Background

Throughout the recent years, the deployment of mobile robots in social environments such as universities, hospitals, restaurants, etc. has noticeably expanded. In such contexts, navigation is no longer considered a purely geometrical task of obstacle avoidance and motion planning; instead, the nature of social interaction introduces new criteria such as safety, comfort, and predictability regarding people (Chen et al., 2025). This domain, commonly referred to as **social robot navigation (SocNav)**, has turned into a hotspot of research due to its relevance in real-world applications.

Despite significant progress in autonomous navigation enabled by cutting-edge powerful hardware and development of modern robotic middleware such as ROS 2, there remains a considerable void in development of robot behaviour in public. Traditional motion planning methods, mainly focus on measurable quantities such as path length, collision avoidance, or time to goal (Nahavandi et al. 2025). Although these metrics are vital for the task, they fall short in capturing qualitative and context-dependant nature of human perception.

Recent studies increasingly support a transition to data-driven assessment methodologies, wherein a robot's performance is evaluated through human subjective judgment rather than exclusively depending on analytical benchmarks. Furthermore, recent advances in machine learning have shifted the focus from rule-based navigation to learning-based methods, where models are trained directly on real-world data. These methods depend heavily on large datasets that accurately reflect human behaviour and variation in the environment. However, the shortage of such datasets remains a major problem, limiting the ability of learned navigation policies to generalise (Xiao et al., 2022).

In addition to behavioural complexity, real-world social environments create a lot of uncertainty due to changing human movement, obstructions, and messy layouts. These factors greatly challenge perception and decision-making systems, forcing robots to continuously change their navigation strategies. As Kruse et al. (2013) noted, socially aware navigation needs to combine both spatial and temporal reasoning to ensure acceptable behavior in human settings. This highlights the need for collecting real-world, high-quality data that captures these complexities, which cannot be fully replicated in simulation-based environments

1.2 Problem Statement

The main prerequisite of developing data-driven standard for SocNav is availability of diverse and high-fidelity data base. Current existing datasets in mobile robotic – simulated/collected in social environments - suffer heavily from various aspects (Kaggle, 2026):

- *Environment*: data is collected in controlled or static environments with limited variability.
- *Comprehensiveness*: lack of multi-modal sensor integration; an ideal dataset must include various values such as visual and geometric perception, state estimation, motion/orientation, and proximity sensing.
- *Temporality*: all collected data must include temporal and contextual information. In a multi-input system one of the major technical challenges is to synchronise the input data. In high-paced

navigation even milliseconds can be important.

- *Setup*: most dataset rely heavily on fixed recording setups/low-robustness mobile setups, preventing scalable and flexible data acquisition.

These limitations prevent researchers from implementing learned models in the field. In particular, the absence of a platform capable of mobile data collection with high precision across various scenarios deprived us from capturing the dynamic nature of real-world social spaces.

However, building such a system present significant engineering challenge as well. A potential model must be capable of:

- Reliable self-driving in real-world settings
- Robust multi-modal sensor fusion with efficient data synchronisation and recording pipelines
- Computational optimisation to ensure real-time performance

Designing such a system requires a high-level a software and hardware understanding.

1.3 Aim and Objectives

The primary aim of this project is to **design and build a mobile robotic platform for collecting multi-modal datasets for social robot navigation**. In order to achieve this goal, the following objectives were defined:

- 1- Reconstruct and configure a fully functional robotic system
 - Deploy a Clearpath Jackal platform with Ubuntu 22.04 server and ROS 2 Humble
 - Configure sensors, drivers, and system configurations from scratch
- 2- Implement a robust autonomous navigation pipeline
 - Develop SLAM-based mapping
 - Enable Localisation using AMCL
 - Configure and deploy the Nav2 navigation stack
- 3- Design and assembly of the structural mount for the hardware housing and camera array
 - Develop a custom housing for the onboard hardware and computing units
 - Integrate a multi-camera array (8 fisheye cameras) for 360° visual data capture
 - Ensure mechanical stability and suitability for mobile deployment
- 4- Develop a synchronized video recording system
 - Develop a software pipeline for multi-camera video acquisition
 - Ensure temporal synchronisation in video recording
- 5- Integration of camera system and robot platform
 - Ensure temporal synchronisation between Jackal and camera system
 - Design orchestration pipeline for multi-source data capture
 - Support both teleoperated and autonomous data collection mode
- 6- Validate the system through experimental testing in real-world environments
 - Perform data collection in multiple environments with varying conditions
 - Evaluate system stability, synchronization, and recording reliability
 - Identify limitations and areas for future improvement

This approach ensures that the system is not only functionally operational but also aligned with the requirements of future data-driven research in social navigation.

1.4 Key Achievements & Technical Contributions

This dissertation makes several contributions spanning both engineering and research domains:

- *Platform Reconstruction*: Fully reconstructed and operationalized a ROS 2-based robotic research platform, transforming a non-functional baseline into a robust, integrated system through comprehensive hardware reconfiguration and sensor synchronization.
- *Modular Navigation Architecture*: Architected an extensible navigation stack using Nav2, enabling seamless SLAM, precise localization, and autonomous navigation capabilities.
- *Custom Multi-Modal Data Acquisition*: Engineered a specialized mobile platform designed for high-fidelity, synchronized data harvesting, capable of capturing:
 - 3D LiDAR point clouds and environmental occupancy maps.
 - High-precision robot state estimation and trajectory tracking.
 - Multi-perspective visual streams via a custom-fabricated camera ring.
- *SocNav Research Infrastructure*: Delivered a scalable solution to the dataset acquisition bottleneck in social navigation (SocNav) research, facilitating the collection of diverse, real-world trajectory data across complex, dynamic environments.

1.5 Project Scope and Limitations

The fundamental objective of this project is to design and develop a robust mobile data acquisition system, prioritising infrastructure over the training or benchmarking of machine learning models. While this project proposes an operational system architecture and a prototype for future SocNav studies, the current implementation is constrained by the limited volume and diversity of the collected data, as well as the preliminary scope of model training and evaluation.

It is important to emphasise that the primary contribution of this work lies in the development of the data acquisition infrastructure rather than the dataset itself. The collected data serves as a proof of system functionality, while future work will focus on scaling, structuring, and utilising the data for machine learning and benchmarking applications.

1.6 Dissertation Structure

This dissertation is organized into seven chapters, each focusing on a specific aspect of designing, developing, and evaluating the mobile multi-modal data acquisition platform.

Chapter 1 (Introduction) provides the background on social robot navigation, defines the problem, and outlines the goals, objectives, and contributions of the project. It explains why developing a data acquisition platform is important for research in socially aware navigation.

Chapter 2 (Literature Review) looks at the current research in social robot navigation, evaluation methods, and existing datasets. It examines robotic middleware frameworks like ROS 2 and Nav2, emphasizing the challenges of multi-modal data acquisition. The chapter ends by identifying the research gap this project aims to fill.

Chapter 3 (System Overview) offers a broad description of the developed platform, including both hardware and software setups. It introduces the Clearpath Jackal robot, the multi-camera acquisition system, and how different subsystems work together to enable synchronized data collection.

Chapter 4 (System Implementation) goes into the practical development of the system. It covers rebuilding the robotic platform, configuring the network, designing the ROS 2 workspace, implementing the navigation stack, and creating custom orchestration nodes. Additionally, it explains how the multi-camera system was implemented using Jetson devices and NVIDIA DeepStream, along with synchronization methods and system optimization strategies.

Chapter 5 (System Results and Outputs) shares the experimental results of the system. This includes mapping results, localization and navigation performance, and examples of the collected multi-modal data. It also evaluates how well the system performs, including its efficiency and synchronization accuracy between the robot and visual data streams.

Chapter 6 (Discussion) reviews the system from engineering and research angles. It discusses the platform's strengths, the main technical challenges faced during development, its limitations, and how it compares to existing methods. The chapter also suggests potential paths for future improvements.

Chapter 7 (Conclusion) summarizes the project's contributions and reflects on how successfully the developed system met its goals. It highlights the platform's importance as a basis for future research in multi-modal data processing and social robot navigation.

2 Literature Review

2.1 Social Robot Navigation (SocNav)

Social robot navigation is an extension to the traditional navigation methods where they incorporate human-aware parameters in addition to geometrical metrics. Traditional methods mainly focus on how to reach a target by avoiding physical obstacles, while social spaces demand appropriate behaviours such as predictability, social norms, and human comfort in addition.

Francis et al. (2025) propose this notion as a definition of SocNav: “A socially navigating robot acts and interacts with humans or other robots, achieving its navigation goals while modifying its behaviour so the experience of agents around the robot is not degraded or is even enhanced.” This definition highlights that the robot behaviour must align with social norms up to a point that the individuals do not feel the need to change their actions regarding to the unit.

Traditional navigation systems are designed to be responsive to geometrical nature of their surrounding and their goal is to optimize objectives such as time to goal and collision avoidance. Examples of these systems are those implemented using ROS Navigation Stack or its successor Nav2. While these methods are effective in less human-centred places such as automated manufacturing sites, they suffer from inherent human-aware protocols which may be technically correct but socially inappropriate.

Observing the system from a mathematical perspective, the whole challenge of robot navigation can be considered an optimisation problem; in such context the objective is to compute a trajectory for the robot that minimises a cost function based on environmental constraints. The general form of this problem is:

$$\pi^* = \underset{\pi}{\operatorname{argmin}} \int_0^T C(x(t), u(t)) dt \quad (\text{Equation 1})$$

In Equation 1, π represents the trajectory, $x(t)$ the robot state, $u(t)$ the control input, and C the cost function incorporating factors such as obstacle proximity and path efficiency. While this formulation is effective for geometric navigation, it does not inherently account for social factors such as human comfort or interaction dynamics (Kruse et al., 2013).

To address this challenge, researchers have proposed various approaches such as Proxemics-based methods, where the robots are obligated to maintain safe distances from humans (Hall, 1966), Human trajectory anticipation models (Pellegrini et al., 2009), and Cost-based methods, where social codes are defined into navigation cost functions (Kruse et al., 2013).

Based on Equation 1, social navigation approaches extend the cost function to include human-aware components. For example, proxemic constraints can be incorporated as an additional cost term:

$$C_{total} = C_{geom} + \lambda \cdot C_{social} \quad (\text{Equation 2})$$

Where C_{social} penalises violation of human's spaces and interaction norms. λ controls the effectiveness of public parameters. This is an example of how purely geometrical models have been evolving toward more comprehensive models.

Although throughout the years the field of SocNav has been improved noticeably, there is still no universally accepted framework for evaluating the social behaviour of robots in public places.

2.2 Evaluation Metrics in Social Navigation

Development of new concepts and methods, urges the existence of proper criteria and metrics. Without a metric system, criticising toward a better outcome becomes impossible and eventually the process will fail. Lack of standardised evaluation metrics is among the central challenges in SocNav. Current approaches rely heavily on physical quantities (e.g., distance to human, number of collisions, path efficiency, time to target) without considering social parameters.

Bachiller-Burgos et al. (2025) introduce the concept of All-encompassing Learned Trajectory-wise (ALT) metrics, in which the trajectories are scored based on human surveys rather than predefined hardcoded parameters. This concept is a great opportunity for SocNav since it incorporates context, such as task urgency or environment in addition to the fact that the output of this metric is strictly based on social preferences rather than analytical measurement.

The authors argue that existing metrics lack empirical validation in the way that they fail to reflect the true human perception of robots behaviour. Accordingly, they offer a benchmarking philosophy which is directly derived from human-scored trajectories.

However, in such context, the fidelity and true diversity of the datasets where the metric is being designed based on can be a great concern. Consequently, it can be stated that success of ALT metrics is heavily dependent on the availability of large, diverse, and well-annotated datasets.

2.3 Datasets for Social Navigation

Datasets are valuable assets in various engineering contexts, particularly for training and evaluating machine learning models. Several datasets have been developed in robots' navigation field:

- THOR (Deitke et al., 2020) - indoor simulation environment with annotated objects
- JRDB (Jiang et al., 2020) - multi-modal dataset with 3D LiDAR and video
- SCAND (Karnan et al., 2022) - large-scale dataset for socially compliant navigation
- SACSoN (Hirose et al., 2023) - real-world social navigation dataset

Overall, these datasets provide the users with valuable information, such as sensor data (LiDAR, RGB, depth), human or robot trajectories, and environmental context. However, there are critical areas that they fall short as well:

- Most datasets suffer from lack of directly human-evaluated robot behaviour.
- The environmental context of the task is often simplified or neglected.
- Data is normally collected in fixed, limited variability, and artificial environments.

As highlighted in prior work, there are currently no datasets specifically designed to support learned trajectory-level metrics for social navigation (Bachiller-Burgos et al., 2025). This gap amplifies the sense of need for new methods of data collection, which enable contextually flexible and scalable way of gathering real-world data.

2.4 Robotic Middleware and Navigation Framework

One of the fundamental elements of modern robotic systems is the middleware framework, which serves as an abstraction layer between the operating system and the robot's software applications.

2.4.1 ROS and ROS 2

The Robot Operating System (ROS) has become the de facto standard for robotics development (Quigley et al., 2009). ROS provides modular system design which is based on nodes, topics, and services. Even though ROS was a revolutionary asset for robotics throughout the last two decades, it has some several limitations. These limitations include lack of real-time support, limited scalability, and security constraints.

ROS 2, which is the upgraded version of ROS, addresses above constraints by introducing Data Distribution Service (DDS) for communication, advanced real-time capabilities, and improved modularity and reliability (Maruyama et al., 2016).

Since ROS 1 is approaching end-of-life, it is essential for modern systems to adapt ROS 2.

2.4.2 Nav2 Navigation Stack

Nav2 (Macenski et al., 2020) is the successor of ROS Navigation Stack, which provides a modular framework for enabling mobile robot to reach autonomy. Nav2 includes features such as global and local path planning, localisation, mapping, obstacle avoidance, etc. Additionally, Nav2 can be configured and coupled with various planning algorithms like Dynamic Window Approach (DWA) (Fox et al., 2002),

A* and Theta* planners (Keirsey et al., 1984), and Model Predictive Control (MPC) variants (Quang et al., 2022).

Fundamentally, autonomous navigation modules in robotic systems such as Nav2 have two main segments: Global path planning and local motion control. Global path planner calculates the best path from starting point to the target in a known map. However, since the environment is dynamic and potentially changing in terms of new obstacles and moving objects, local planner generates real-time velocity commands to follow the ideal path and manoeuvring moving obstacles.

From a mathematical perspective, the planning problem can be formulated as a graph search over a discretised environment, where the objective is to minimise a cumulative cost function. For instance, in grid-based planners such as A*, the cost of a node is defined as:

$$f(n) = g(n) + h(n) \quad (\text{Equation 3})$$

where $g(n)$ represents the accumulated cost from the start node to node n , and $h(n)$ is a heuristic estimate of the remaining cost to the goal. This formulation allows efficient exploration of the search space while maintaining optimality under admissible heuristics (Hart et al., 1968; Macenski et al., 2020).

At the level of local control, algorithms like the Dynamic Window Approach (DWA) work in velocity space instead of configuration space. DWA doesn't plan a full trajectory for the robot; instead, it samples acceptable velocity commands and tests them over a short prediction horizon. The function that determines the best control input can be written as:

$$v^* = \operatorname{argmax}_{v \in V} (\alpha \cdot \text{heading}(v) + \beta \cdot \text{clearance}(v) + \gamma \cdot \text{velocity}(v)) \quad (\text{Equation 4})$$

where the terms represent alignment with the goal direction, distance from obstacles, and forward velocity respectively. The α , β , γ weights determine the behaviour of the robot, balancing safety and efficiency (Fox et al., 2002).

Although Nav2 has been a robust and powerful tool for roboticist, it inherently misses the ability to address social navigation prerequisites, reinforcing the necessity for more robust evaluation and learning framework.

2.5 Data Acquisition challenges in Robotics

Acquiring high-fidelity, multi-modal datasets in robotics involves significant challenges. Data collection requires diversity in context and environment, sufficient processing power, and proper temporal system integration. Gathering all these criteria, current data collection setups are limited in flexibility, mobility, and reproducibility.

A central challenge in multi-modal systems is temporal synchronisation, which ensures that data collected from different sensors corresponds to the same physical moment. This can be mathematically expressed as minimising the temporal offset between sensor streams:

$$\Delta t = |t_i - t_j| \rightarrow 0 \quad (\text{Equation 5})$$

Where t_i and t_j denote timestamps from two different sensors. Practically, achieving near-zero temporal offset is difficult due to network delays, processing latency, and independent system clocks, particularly in distributed architectures.

Accordingly, a unified data collection platform which is able to address the above challenges would be the next major milestone in SocNav. This platform must:

- Operate in various environments and scenarios
- Record synchronized multi-modal data
- Have enough processing power for further system extensions
- Operate autonomously or human-controlled

2.6 Research Gap

Based on the literature, several key gaps can be identified:

- 1- Lack of trajectory-level datasets with human evaluation: Current datasets are inadequate for training learned social navigation metrics.
- 2- Limited contextual and geometrical diversity: Most datasets are collected in fixed environments, restricting variability.
- 3- Absence of mobile, scalable acquisition systems: There isn't a widely used platform for gathering data from various real-life situations.
- 4- Separation between navigation systems and data collection pipelines: Many systems focus on navigation or data recording independently, rather than integrating both.

2.7 Summary

This chapter reviewed the state of social robot navigation (SocNav), evaluating existing methods for benchmarking and system architecture. The assessment reveals that while algorithm development and simulation environments have progressed, research is hindered by the absence of data-driven evaluation methods. A primary constraint is the lack of specialized, mobile data acquisition platforms capable of capturing diverse, multi-modal data in real-world environments. Consequently, this dissertation introduces a mobile robotic platform designed to facilitate the collection of synchronized datasets, providing a foundational tool for the advancement of future social navigation studies.

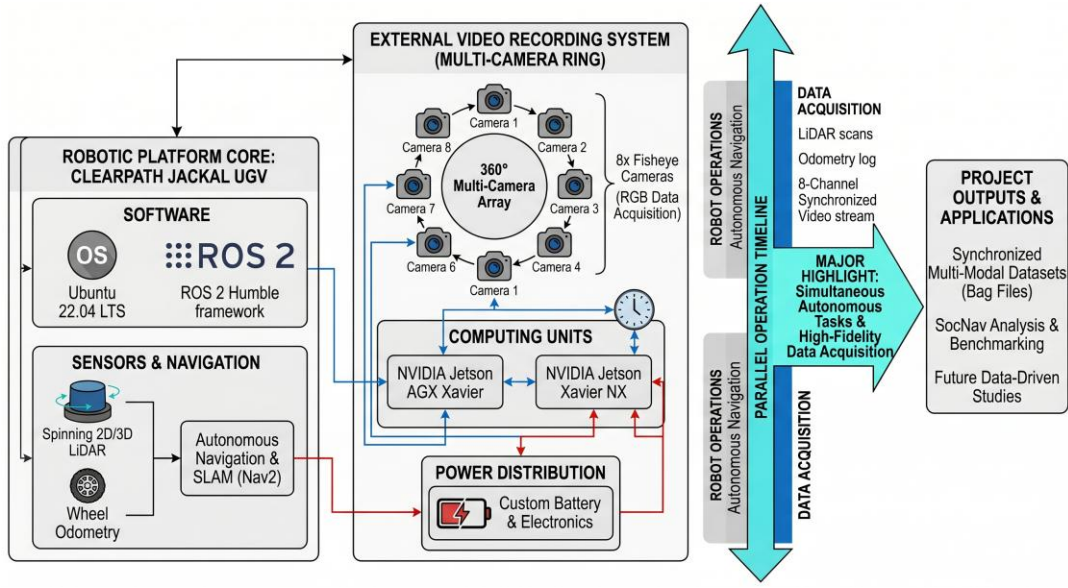


Figure 3.1: High-Level System Architecture

3 System Overview

3.1 System Overview

The system developed in this project is a **mobile multi-modal data collecting platform**, designed to support SocNav research. The architecture integrates a ground robot with an external video recording system, which enables comprehensive data acquisition in various environments and in different modes.

At the core of the platform, there is a **Clearpath Jackal** Unmanned Ground Vehicle (UGV), which is configured with Ubuntu 22.04 and ROS 2 Humble. The robot is equipped with a full range of sensors, such as LiDAR and odometry which enables the robot for autonomous navigation, mapping, etc.

On the other hand, since SocNav depends heavily on RGB information of humans in the environment, an external 360 multi-camera ring is designed and mounted on the top of Jackal. This system consists of necessary computing units (Jetson Xavier and AGX devices), battery and electronics for powering components, and most importantly 8 fisheye cameras.

A major highlight of this system is the ability to perform autonomous tasks parallel to high-fidelity data acquisition, while prior systems tend to do one of these two actions at a same time. This feature can be the basis for future SocNav analysis and benchmarking.

3.2 Hardware Architecture

The hardware design of this project is composed of two subsystems: the robotic platform and the external RGB data acquisition system.

3.2.1 Robotic Platform

Clearpath Jackal serves as the ground robot in this system. Jackal is considered a lightweight, fast, and user-friendly robot, which is capable of outdoor navigation due to its durable chassis design and relatively big wheels.

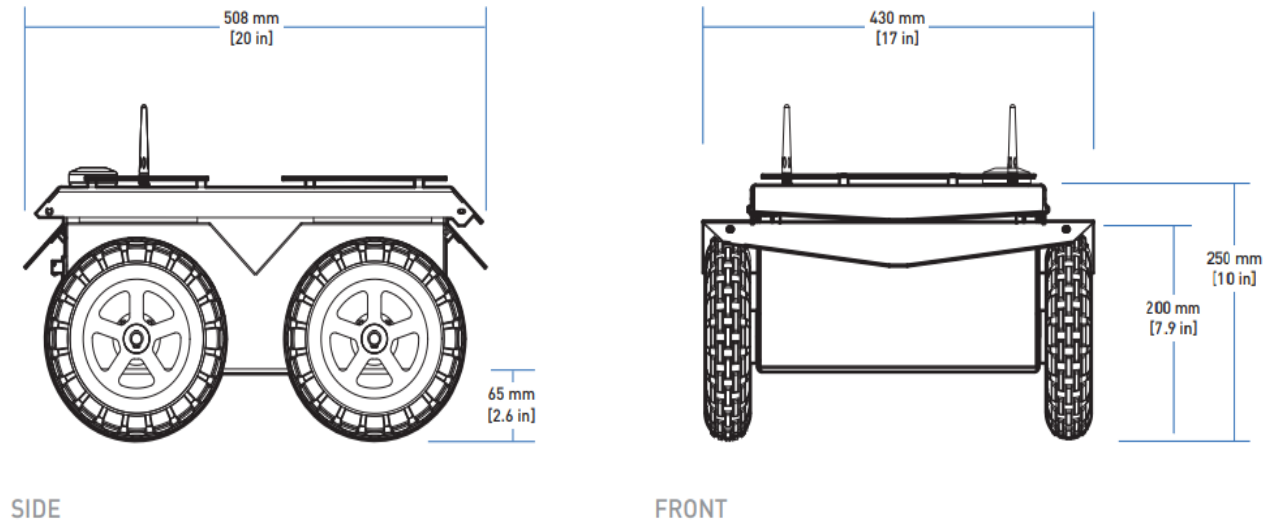


Figure 3.2.1: Clearpath Jackal Robotic platform (Clearpath Robotics, 2026)

Jackal comes with processing power enough for multiple sensor integration, running ROS 2 nodes, and handling complicated autonomous navigation tasks other than many other peripheral tasks. Additionally, Jackal has SICK TIM551 2D LiDAR plus odometry unit and IMU for navigation. Figure 3.2.1 demonstrates Jackal's physical features.

3.2.2 Multi-Camera Acquisition System

To capture visual context, a cylindrical array of eight fisheye cameras is mounted on the robot. Each one of these cameras offers a field of view of 170 degrees and they are mounted in the way that each camera has neighbouring cameras 45 degrees apart on both sides. Figure 3.2.2-a shows this structure which enables the user to have a nearly complete vision to the surrounding of the robot.

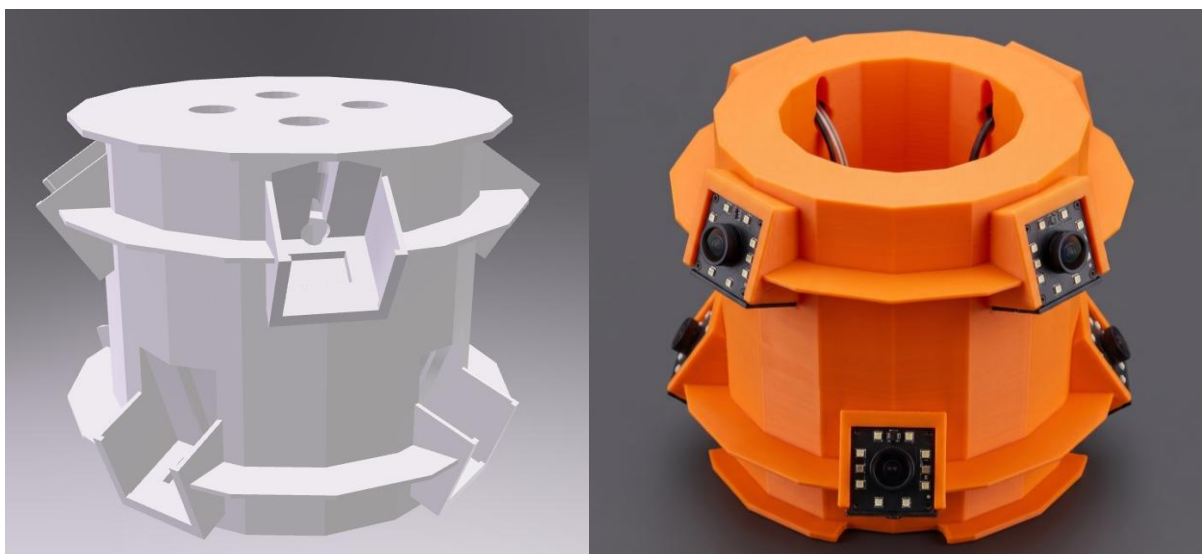


Figure 3.2.2-a: The camera array designed for vision system, able to capture 360- videos.

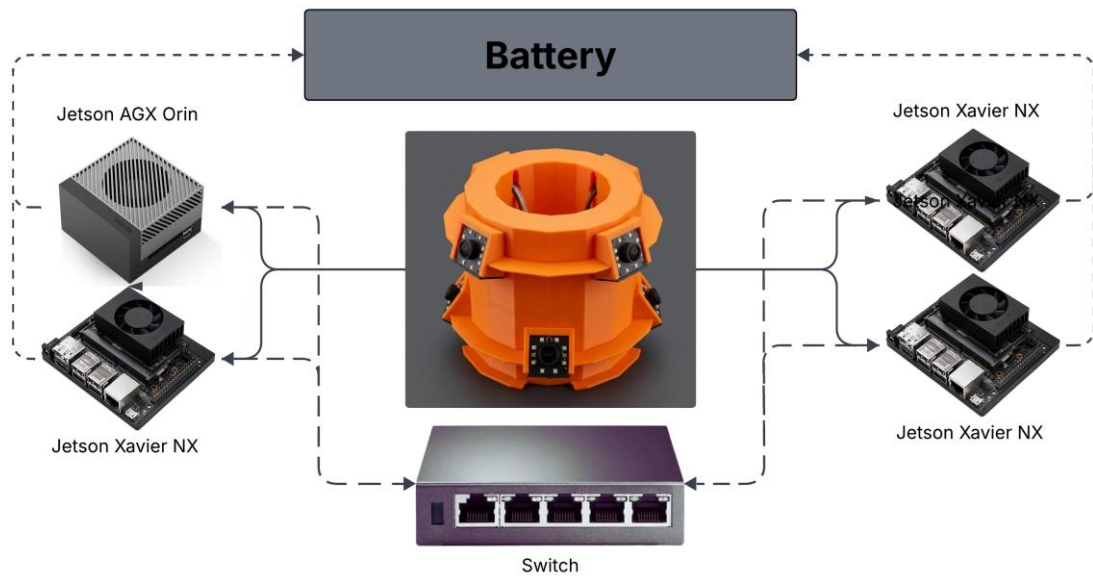


Figure 3.2.2-b: Camera system architecture

In order to support the cameras four embedded computing devices are provided. A Jetson AGX and three Xavier units, where two cameras are connected to each one of them. In order to be able to synchronise video recordings, these computers are connected together through a network switch. Finally, the entire system is powered via an external battery and voltage regulation components. Adding an external power source, makes sure that the Jackal's battery won't be overloaded and therefore the operational time of the entire system will be increased. Figure 3.2.2-b illustrates the design of the camera system.

In order to be able to mount this setup on the Jackal with minimum weight and maximum rigidity, Aluminium profiles are used. These profiles are bolted perpendicular to a steel plate on Jackal's top. Also, 3D-printed parts were made to attach components to the profiles with the help of DIN rails. Finally, the battery is directly mounted on the Jackal, stabilised with strap. Figure 3.2.2-c shows the robot's final shape.



Figure 3.2.2-c: Jackal in final state. The orange parts are 3D printed, using PLA.

3.3 Software Architecture

The software of this project is built on ROS 2, and it follows a modular design. The modularity allows for future extension and improvement of the system where new modules can be easily added and configured without damaging the other segments.

At a high level, there are five major group of nodes designed for this project:

- Perception Nodes (e.g., LiDAR processing)
- Localisation and Mapping components
- Navigation Modules (Nav2)
- Data Recording processes
- Orchestration Nodes (Communicating between the camera system and Jackal)

Communication between elements of the system is achieved through ROS topics, services, and actions.

The interaction between nodes and the data communicating between them in this system is based on ROS topics. Most nodes publish their data in a reasonable frequency and on the other hand, the nodes that need that information subscribe to the topic. For instance:

- LiDAR data is published to a scan topic
- navigation commands are sent via velocity topics
- localization outputs are broadcast through pose topics

Table 3.3: Key ROS Topics and Definitions

Topic Name	Category	Description
/j100_0796/sensors/lidar2d_0/scan	Sensor	The raw laser scan data output by the SICK 2D lidar.
/j100_0796/cmd_vel	Navigation	Velocity commands (linear/angular) sent to the Jackal base to move the robot.
/j100_0796/tf	Transforms	Dynamic coordinate frames describing the relationship between odom, base_link, and lidar2d_0_laser.
/j100_0796/tf_static	Transforms	Static coordinate frames for the robot's hardware configuration.
/j100_0796/amcl_pose	Localization	The estimated pose of the robot published by AMCL for monitoring/debugging.
/j100_0796/map	Localization	The occupancy grid map currently used for navigation/localization.
/map	Mapping	The occupancy grid map used specifically during the live SLAM building phase.
/goal_pose_hijacked	Orchestration	The entry point where RViz 2D Goals are intercepted by your script before processing.
/j100_0796/goal_pose	Navigation	The validated, "clean" goal pose published to Nav2 once the orchestrator is ready.
/j100_0796/initialpose	Localization	The initial pose estimate sent to AMCL to start the localization process.
/j100_0796/robot_description	System	The URDF/Robot model definition of the Jackal.

The system supports multiple operational modes, each designed for a specific purpose. Based on the nature of the mission, Jackal may need to work autonomously or manually. For example, in mapping mode, the operator has to move the robot around via the controller so the robot can collect environmental data and build the map. In localisation mode, the robot estimates its position within a known map using AMCL. On the other hand, Navigation mode doesn't need the oversight from the operator since the robot autonomously navigates to a given target in a known map. More advanced modes include autonomous or manual recording where the Jackal is able to navigate while gathering synchronised data.

3.4 System Summary

The architecture in this chapter gives mobile data collection a modular and flexible base. The system lets the user to collect data in real-world settings that can be scaled up and synced by combining a robotic platform with an external sensing system and a flexible software framework.

4 System Implementation

4.1 System Reconstruction

The implementation of the proposed data collection system started with a full system reconstruction of the Jackal Clearpath. Initially the robot was delivered with a WD Blue SA510 SATA SSD, which due to the critical problem of firmware- posted by the manufacturer- faced a permanent failure (Clearpath Robotics, 2026). The robot initially was configured with Ubuntu 20.04 and ROS 1 Noetic, however because of the loss of the internal SSD and the absence of a recoverable system image, the entire software environment had to be rebuilt from the first principles.

This phase of reconstruction was not just a restoration step; it was also a chance to redesign the system on purpose. This made it possible to move to a modern robotics middleware architecture and set up a controlled, repeatable baseline for future system integration. Switching to ROS 2 Humble, in particular let the system take advantage of better communication paradigms (DDS-based middleware), lifecycle-managed nodes, and better support for distributed robotic systems.

4.1.1 Clearpath-Specific Operating System Deployment

The use of a Clearpath-customized Ubuntu 22.04 LTS image was a key part of the reconstruction process, as stated in the official Clearpath Robotics installation documentation. This image is already set up to work with the Jackal hardware platform, which is different from a regular Ubuntu installation. This image includes:

- Network interface configuration for the internal robot subnet (e.g., LiDAR and onboard devices)
- Predefined system users and permissions
- Hardware-level dependencies and drivers required for the Jackal base platform
- Integration hooks for Clearpath-provided ROS 2 packages

This customized operating system works with the robot's built-in systems right away and makes it a smooth experience compared to set up the hardware manually. It also makes sure that it works with Clearpath's supported ROS 2 deployment model, especially when it comes to communicating to sensors and actuators on board.

4.1.2 Initial System Configuration and Installer Execution

Upon first boot of the Clearpath-provided Ubuntu 22.04 image, the system is accessed using default credentials defined by the vendor. As part of standard security practice, the initial step involves updating the administrator password using the Linux `passwd` utility.

After the login process and change of the password, a specific installation script, provided by Clearpath, needs to be executed:

```
bash -e clearpath_computer_installer.sh
```

This script is a critical component of the deployment process. Instead of manual ROS installation and individual package setup, the script performs a full system provisioning, including:

- Installation of ROS 2 Humble and required middleware dependencies
- Deployment of Clearpath-specific ROS 2 packages for the Jackal platform
- Configuration of hardware interfaces (motor drivers, onboard controllers)
- Setup of system services and environment variables
- Integration of sensor communication layers (e.g., LiDAR networking)

The installation process is interactive and it needs inputs from the operator such as robot's model and desirable packages to install.

This automated provisioning method makes sure that the system matches Clearpath's supported architecture. It provides a verified baseline where all low-level hardware interfaces are already set up correctly.

4.1.3 Internal Communication

A critical aspect of the Jackal system is its segregated network architecture, which separates internal robot communication from external connectivity. This is implemented using Netplan, with a bridge interface that connects multiple physical Ethernet ports into a single logical interface. The primary Netplan configuration is shown on the next page.

This configuration establishes a bridge interface (*br0*) that aggregates the physical interfaces (*eno1*, *enp1s0*) into a unified internal network. The bridge is assigned a static IP address (*192.168.131.1/24*), which acts as the central communication gateway for onboard devices such as LiDAR , embedded controllers, and additional ethernet connected peripherals.

To enable deterministic communication with external compute modules, an additional static IP configuration was applied to one of the physical Ethernet interfaces within the Netplan configuration. Specifically, the *eno1* interface was assigned an address within the *192.168.5.0/24* subnet.

```

network:
  version: 2
  renderer: networkd

ethernets:
  eno1:
    addresses: [192.168.5.88/24]
    dhcp4: no
    dhcp6: no
  enp1s0:
    dhcp4: no
    dhcp6: no

bridges:
  br0:
    interfaces: [eno1, enp1s0]
    addresses: [192.168.131.1/24]
    dhcp4: no
    dhcp6: no
    parameters:
      stp: false
      forward-delay: 0

```

This configuration establishes a dedicated point-to-point network segment for communication with an external processing unit, namely the Jetson AGX Orin, which is used as part of the camera and recording subsystem. By assigning a static IP address and disabling DHCP, the connection remains stable and predictable, which is essential for triggering remote commands (e.g., via SSH) and synchronising recording pipelines during mission execution. The use of a separate subnet (*192.168.5.XX*) also ensures logical isolation from the internal robot network (*192.168.131.XX*), preventing interference with onboard sensor communication while maintaining a reliable high-bandwidth link to the external vision system.

4.1.4 External Connectivity via Wi-Fi

While the internal network is isolated and statically configured, external connectivity (e.g., internet access, remote SSH, ROS communication with a laptop) is handled separately using a Wi-Fi interface managed by NetworkManager.

Since the proposed system is a mobile platform, the external connection source must be mobile as well. Accordingly, at this point, the Wi-Fi is configured to connect to a smartphone's hotspot. This configuration is defined in a dedicated Netplan file:

```

network:
  version: 2
  renderer: NetworkManager

wifis:
  wlp2s0:
    dhcp4: true
    access-points:
      "Xiaomi 14T":
        password: "*****"

```

In this system, the Wi-Fi interface is configured to use DHCP rather than a static IP address because it operates within a dynamic, externally managed network environment (e.g., mobile hotspots or public/private access points). Unlike the robot's internal networks- where all devices are known in advance and can be assigned fixed addresses- the external Wi-Fi network is controlled by a router (such as a smartphone hotspot) that dynamically allocates IP addresses from a predefined pool. Attempting to manually assign a static IP on this interface can lead to IP conflicts, incorrect gateway or DNS configurations, and loss of connectivity if the chosen address falls outside the router's allocation range. Furthermore, mobile hotspots frequently change network parameters (subnet, gateway, lease range), making static assignment unreliable across sessions. By enabling DHCP, the robot automatically receives valid network configuration parameters (IP address, subnet mask, gateway, DNS) at connection time, ensuring consistent and robust internet access and remote connectivity without manual reconfiguration.

4.1.5 Connectivity Validation

During various processes, the user may need to check the state of connection for the Jackal. Typically, the following commands are used for this purpose:

```
ip a          # verify interface configuration
ip route      # inspect routing tables
ping <device_ip> # test connectivity (internal/external)
```

4.2 Workspace and Package Architecture

Following the successful rebuilding of the base system and middleware environment, a well-organized ROS 2 workspace was set up to support modular development, manage configurations, and integrate systems. The workspace was created at:

```
/home/administrator/jackal_clearpath_ws/ros_ws
```

This workspace follows the standard colcon-based ROS 2 structure. All source packages are located in the *src/* directory and are compiled into separate build and install spaces. This setup allows for reproducibility, keeps dependencies separate, and supports scalable development across various subsystems.

At a high level, the project is organised into three primary layers:

- 1- ROS-bag repository, where the collected data from Jackal's sensor are archived (Outside ROS workspace).
- 2- Jackal Clearpath workspace (packages, nodes, launch system)
- 3- Video repository, where the camera system recordings are transferred and saved.

A simplified representation of Jackal Clearpath workspace structure is shown below:

```
jackal_clearpath_ws/
├── ros_ws/
│   ├── src/
│   │   └── jackal_custom_config/
│   │       ├── config/
│   │       │   ├── active/
│   │       │   └── legacy/
│   │       ├── launch/
│   │       │   ├── sensors/
│   │       │   └── legacy/
│   │       ├── scripts/
│   │       │   └── startup/
│   │       ├── rviz/
│   │       ├── docs/
│   │       ├── CMakeLists.txt
│   │       └── package.xml
│   ├── build/
│   ├── install/
│   └── log/
├── maps/
├── config/
└── netplan/
```

- **Custom Package: jackal_custom_config**

The core of the system is contained within a single custom ROS 2 package:

```
jackal_custom_config
```

This package serves as the central layer for the robot, integrating:

- Navigation and localization parameters
- Sensor configurations (e.g., LiDAR, cameras)
- Launch orchestration for different operational modes
- Custom Python nodes for mission execution and data recording

Unlike standard ROS packages, this package is specifically designed as a system configuration and management package. It focuses on system integration and coordinating the pipeline as a reflection of project's nature.

- **Configuration Management (config/)**

The *config/* directory holds all YAML-based parameter files that configure ROS 2 nodes during runtime. A significant design choice was to create an *active/* and *legacy/* separation:

- *active/*: contains the current, validated configurations used in deployment
- *legacy/*: stores previous versions for rollback, comparison, and debugging

This setup allows for controlled changes during tuning (like adjusting costmaps, AMCL parameters, or camera settings) without losing the previous working configurations.

- **Launch System Architecture (launch/)**

The `launch/` directory outlines the system's runtime setup, using ROS 2 Python-based launch files. These files handle:

- Instantiating nodes
- Applying parameter files
- Performing topic remapping
- Enforcing namespace consistency (/j100_0796)

This structure further modularised into sensors launch files and root launch files.:

- `sensors/`: sensor-specific launch files (e.g., LiDAR, cameras)
- root launch files: high-level system modes (SLAM, localization, navigation)

- **Custom Runtime Nodes (scripts/)**

The `scripts/` directory contains Python-based ROS 2 nodes that implement custom orchestration logic, which is a crucial part of this project. These scripts go beyond standard ROS functions and enable:

- Mission-level coordination
- Recording pipeline automation
- External system communication (e.g., Jetson via SSH)

An example of why scripts are vital is the `goal_mission_orchestrator.py` script. This is the main link between camera system and Jackal. When a goal is sent to Jackal's autonomous system, this scripts send a message to camera system to start recording. At the same time, it starts recording the ROS-bag to collect Jackal's sensor as well. Finally, when the target is reached, the scripts switch off the recordings and move to stand by position to receive the next mission.

- **System-Level Configuration (config/ outside workspace)**

In addition to ROS-specific configuration, a top-level `config/` directory is used for system-level settings. This separation is important because these files:

- Configure hardware and networking, not ROS nodes
- Are applied at the operating system level
- Must persist independently of ROS package builds

It worth mentioning that the `netplan/` directory in `config/`, is just a copy of the original directory. Changing the connection settings in the copy version, doesn't affect the system-level configuration.

Table 4.2 shows a summary of the description for directory architecture of Jackal.

Table 4.2: Directory Structure Description

Directory	Purpose
launch/	ROS 2 launch files defining system runtime composition
config/	YAML parameter files for navigation, localization, and sensors
scripts/	Custom ROS 2 nodes for orchestration and recording control
maps/	Stored SLAM-generated maps (YAML + PGM)
ros_ws/	ROS 2 workspace containing all buildable packages
config/netplan/	System-level network configuration

4.3 Configuration Architecture

This system is critically designed based on YAML-based configuration files to control and customise all the subsystems, including SLAM, localisation, navigation, and sensors launching. In ROS 2, parameters are externalised from source code and injected into nodes at runtime via launch files. This approach enables rapid tuning, reproducibility, and modularity, allowing system behaviour to be modified without recompilation.

Within this project, configuration files are organised under the *config/active/* directory of the *jackal_custom_config* package, ensuring that only validated parameter sets are used during deployment. The parameters are structured hierarchically under the robot namespace (*j100_0796*), which ensures consistency across all nodes and prevents conflicts in multi-node communication.

4.3.1 SLAM Configuration (SLAM Toolbox)

slam_toolbox parameter file is used to form the SLAM subsystem, which defines both mapping behaviour and scan matching performance. A representative excerpt is shown below:

```
slam_toolbox:
  ros__parameters:
    map_frame: map
    odom_frame: odom
    base_frame: base_link
    scan_topic: /j100_0796/sensors/lidar2d_0/scan

    resolution: 0.05
    max_laser_range: 20.0
    map_update_interval: 2.0

    use_scan_matching: true
    do_loop_closing: true

    minimum_travel_distance: 0.10
    minimum_travel_heading: 0.10
```

These parameters essentially define how Jackal constructs a 2D occupancy grid map from LiDAR data. Some of the key technical aspects are as follows:

- Frame definitions (*map* $\Rightarrow$ *odom* $\Rightarrow$ *base_link*) ensure consistency with the TF tree and enable proper transformation of sensor data.
- The *scan_topic* is explicitly remapped to the name-spaced LiDAR topic, ensuring integration with the hardware driver.
- The resolution (*0.05 m*) defines the granularity of the generated map, directly impacting memory usage and map fidelity.
- *use_scan_matching* and *do_loop_closing* enable graph-based SLAM optimisation, allowing the system to correct accumulated drift through loop closure detection.

More advanced parameters (e.g., correlation search space, loop closure thresholds) were tuned to balance mapping accuracy vs computational load, particularly given the CPU constraints of the onboard system.

4.3.2 Localisation Configuration (AMCL)

In order to estimate the robot's pose, Adaptive Monte Carlo Localisation (AMCL) algorithm is used.

```
j100_0796:
  amcl:
    ros__parameters:
      global_frame_id: map
      odom_frame_id: odom
      base_frame_id: base_link

      scan_topic: /j100_0796/sensors/lidar2d_0/scan

      min_particles: 50
      max_particles: 150

      update_min_d: 0.20
      update_min_a: 0.20
```

Key technical considerations include:

- Particle bounds (*min_particles*, *max_particles*) define the trade-off between localization accuracy and computational cost.
- *update_min_d* and *update_min_a* control how frequently the filter updates based on robot motion, reducing unnecessary computations during small movements.
- The scan topic alignment ensures consistency with the SLAM and navigation subsystems, enabling seamless transition between mapping and localization modes.

Additionally, the map server is configured within the same YAML:

```
map_server:
  ros__parameters:
    yaml_filename: "/home/administrator/jackal_clearpath_ws/maps/map_306.yaml"
```

This section allows the system to use a prebuilt map, ensuring that the localisation operates on a fixed global reference.

4.3.3 Navigation Stack Configuration (Nav2)

The navigation subsystem was implemented using the ROS 2 Nav2 framework, with a configuration designed to support multiple planning and control algorithms during data collection experiments. Rather than fixing a single navigation method, the system allows switching between different global planners (e.g., *A**, *Dijkstra/NavFn*) and evaluating their effect on robot behaviour and recorded data.

At the global planning level, a grid-based planner was configured with a frequency of 5 Hz, which was sufficient for the operating environment while keeping CPU usage low:

```
planner_server:
  ros__parameters:
    expected_planner_frequency: 5.0
    planner_plugins: ["GridBased"]

    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: false
      allow_unknown: true
```

In this setup, the planner runs in Dijkstra mode (*use_astar: false*), prioritising robustness and full costmap exploration. However, the configuration allows switching to *A** when faster, heuristic-based path generation is required. This flexibility was important during testing, as different planners produced noticeably different path shapes and traversal behaviour, which directly affected the timing and consistency of recorded datasets.

The local control layer was configured using the DWB (Dynamic Window Approach) controller, running at a higher frequency of 10 Hz to ensure responsive motion:

```
controller_server:
  ros__parameters:
    controller_frequency: 10.0
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "dwb_core::DWBLocalPlanner"
```

The controller parameters were tuned to match the physical limits of the platform:

```
FollowPath:
  max_vel_x: 0.35
  max_vel_theta: 1.2
  acc_lim_x: 0.5
  acc_lim_theta: 0.3

  vx_samples: 20
  vtheta_samples: 20
  sim_time: 1.5
```

These values define the velocity search space and trajectory simulation horizon. Increasing `vx_samples` and `vtheta_samples` improved trajectory quality but also increased computational load, so the selected values represent a balance between smooth motion and real-time performance.

Trajectory evaluation is performed using multiple critics:

```
critics:
  - RotateToGoal
  - Oscillation
  - BaseObstacle
  - GoalAlign
  - PathAlign
  - PathDist
  - GoalDist
```

In practice, the most influential weights were *PathAlign*, *GoalAlign*, and *PathDist*, which were tuned to prioritise stable path tracking over aggressive shortcutting. This was particularly important for data collection, where consistent and repeatable motion with less vibration and more stability is preferred over minimal path length.

Costmap configuration played a significant role in navigation performance. Both global and local costmaps use a resolution of 0.05 m, which matches the SLAM map resolution:

```
global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 5.0
      resolution: 0.05
```

```
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 10.0
      width: 3
      height: 3
      resolution: 0.05
```

An inflation radius of 0.55 m provided a safe clearance margin while still allowing navigation through moderately narrow spaces. Larger values were tested but resulted in blocked paths in constrained environments.

```
inflation_radius: 0.55
cost_scaling_factor: 3.0
```

From an implementation perspective, the key point is that navigation was not treated as a fixed pipeline. Instead, parameters such as planner type (*A** vs *Dijkstra*), controller frequency (*10 Hz*), velocity limits (*0.35 m/s*), and inflation radius (*0.55 m*) were deliberately tuned and adjusted to support different experimental conditions. This allowed the system to generate varied motion profiles, which was essential for collecting diverse and representative datasets.

4.3.4 LiDAR Integration

The LiDAR sensor in this project is a SICK TIM 551 series scanner, set up via the `sick_scan_xd` driver and connected through Ethernet. This unit is a 2D time-of-flight laser scanner for mobile robotics purpose, giving planar measurement services which are reliable in distance measurement regarding mapping and navigation. The sensor features a 270° field of view for broad environmental scanning, and the range of distance measurements is fully compatible with the `max_laser_range` parameter set in the SLAM and navigation stack, capped at 20 meters.

The scan data is published as a standard `sensor_msgs/LaserScan` message on the `/j100_0796/sensors/lidar2d_0/scan` topic, thus ensuring compatibility with SLAM Toolbox, AMCL, and Nav2 costmaps. The sensor utilises a dedicated internal network (192.168.131.20) and uses TCP/IP (port 2112) for communication to ensure stable and low latency in data transmission.

This sensor is configured via a dedicated YAML file:

```
sensors:
  front_lidar:
    driver: sick_scan_xd
    hostname: 192.168.131.20
    port: 2112
    topic: scan
```

This configuration defines:

- Ethernet communication parameters (IP and port)
- ROS topic naming for sensor output
- Frame alignment with the TF tree

Additionally, TF publishing is disabled at the driver level:

```
tf_publish_rate: 0.0
```

This prevents conflicts with the main TF tree, ensuring that all transforms are centrally managed.

4.3.5 Robot-Level Configuration

At the system level, a `robot.yaml` file defines global properties such as robot's hostname, IP, namespace, ROS parameters, and physical links attached to the robot and their exact position and size. Below is a small section of this file from Jackal:

```
serial_number: j100-0796
version: 0
system:
  hosts:
    - hostname: cpr-j100-0796
      ip: 192.168.131.1
  ros2:
    namespace: j100_0796
platform:
  attachments:
    - name: rear_fender
      type: j100.fender
      rpy: [0.0, 0.0, 3.1415]
```

4.4 Node Implementation and Launch System

The system is comprised of several ROS 2 nodes for navigation, sensor processing, and data logging. This work introduced a series of python-based nodes that provide mission-level orchestration and integration of Jackal platform with an external recording system (Jetson AGX Orin), in addition to the regular Nav2 and sensor nodes.

One of the highlight architectural design principles is the dichotomy between control and orchestration. The navigation nodes (Nav2) are strictly for the task of motion generation (`/cmd_vel`), while custom nodes function at a higher level and are used for orchestrating various tasks like recording, dealing with goals, and interacting with the outside world. This prevents added functionality from undermining the navigation stack's stability.

4.4.1 Orchestration Node

The primary custom nodes in this project are the group of mission orchestrators. There are mainly three different nodes defined in order to lead the chain of actions for data recording of the robot and camera system. The difference between these nodes is the working mode of Jackal; whether it is driving autonomously or being controlled by teleoperation or finally the state that the recording system collects the data, regardless of Jackal's movement, until the user commands to stop it.

A comprehensive data gathering system must be able to collect the data in all states. Based on this, the three types of defined node for orchestration enable data collection in these scenarios:

- **Autonomous Navigation:** This program is responsible for collecting data while the robot is moving autonomously, using various navigation algorithms.
- **Teleoperation Mode:** There are some scenarios that the researchers want to drive the robot manually and record people's reaction. Manual control can be caused by different intentions such as safety or recording missions with intentional failure which can not be directly programmed on autonomous systems.
- **Recording regardless of movement:** In some cases, the data collection must be done regardless of robot's movement or the state of targets achievement. For instance, collecting data while the robot is waiting in a queue or an elevator can be among these cases.

- **Autonomous Navigation Mode**

This node is responsible for collecting data while the robot is navigating autonomously using the Nav2 stack. The key functionality of this node is to intercept navigation goals, trigger the recording pipeline, and coordinate execution timing.

A simplified structure of this node is shown on the next page. Instead of directly using the standard `/goal_pose` topic from RViz, a remapped topic named: `/goal_pose_hijacked` introduced. This allows the node to act as an interception layer before the goal is forwarded to `/j100_0796/goal_pose`.

```

class GoalMissionOrchestrator(Node):
    def __init__(self):
        super().__init__('goal_mission_orchestrator')

        self.subscription = self.create_subscription(
            PoseStamped,
            '/goal_pose_hijacked',
            self.goal_callback,
            10
        )

    def goal_callback(self, msg):
        self.start_recording()
        self.send_navigation_goal(msg)

```

The full execution pipeline implemented in this mode is:

1. Receive goal from RViz (/goal_pose_hijacked)
2. Trigger Jetson recording via SSH
3. Apply delay (~10 seconds) to ensure camera readiness
4. Start ROS bag recording of all topics
5. Send goal to Nav2 (navigate_to_pose action)
6. Wait for navigation completion (action result)
7. Stop rosbag recording
8. Send SSH stop command to Jetson

- **Teleoperation Mode**

In this mode, the robot is controlled using:

```
ros2 run teleop_twist_joy teleop_node
```

The orchestration node (teleop_record_orchestrator.py) operates independently and only handles recording triggers.

Key design constraint:

- The node does not interfere with /cmd_vel
- Teleoperation remains the sole velocity publisher

Recording is triggered via controller input:

```

START → SSH trigger + rosbag start
STOP → rosbag stop + SSH stop

```

- **Recording Regardless of Movement**

For this purpose, a third node (`keyboard_record_orchestrator.py`) was implemented, allowing manual control of the recording pipeline via keyboard input:

```
s → start recording
x → stop recording
q → exit node
```

4.4.2 Launch File Configuration

To this end, the system uses ROS 2 Python-based launch files to initialise and coordinate all components, including navigation, sensors, and orchestration nodes. Instead of using one large launch file, the project uses a modular launch approach where separate subsystem gets launched.

Inherently, ROS 2 launch system allows nodes to be defined, parameterised, and remapped at the time of launching. In practice, launch files are complex and include:

- Loading YAML configuration files (navigation, localization, sensors)
- Applying namespace (`j100_0796`)
- Remapping topics (e.g., `/tf`, `/goal_pose`)
- Starting multiple nodes together (Nav2 stack, lifecycle managers, sensors)

The system is launched in multiple stages rather than a single command. A typical autonomous recording workflow is:

```
# Terminal 1: LiDAR Start Up
ros2 launch jackal_custom_config sick_tim_custom.launch.py

# Terminal 2: Localisation Start Up
ros2 launch jackal_custom_config nav2_localization_jackal.launch.py

# Terminal 3: Navigation Start Up
ros2 launch jackal_custom_config nav2_navigation_jackal.launch.py

# Terminal 4: Orchestration node Start Up
ros2 launch jackal_custom_config goal_mission_orchestrator.launch.py

# External Computer (Connected to the Jackal - Same $ROS_DOMAIN)
rviz2 --ros-args \
  -r /goal_pose:=/goal_pose_hijacked \
  -r /tf:=/j100_0796/tf \
  -r /tf_static:=/j100_0796/tf_static
```

Within the `jackal_custom_config` package, various launch files are developed for different purposes. However, below is the list of the most important ones that are used during data collection sessions, also Table 4.4.2 demonstrates the responsibility of each of these launch files.

Table 4.4.2: Launch Components and Responsibilities

Launch File	Purpose
<code>nav2_navigation_jackal.launch.py</code>	Starts full navigation stack
<code>nav2_localization_jackal.launch.py</code>	Runs AMCL + map server
<code>slam_toolbox_jackal.launch.py</code>	Runs SLAM mapping mode
<code>goal_mission_orchestrator.launch.py</code>	Starts orchestration node
<code>sick_tim_custom.launch.py</code>	Initialises LiDAR driver

```

launch/
├── nav2_navigation_jackal.launch.py
├── nav2_localization_jackal.launch.py
├── slam_toolbox_jackal.launch.py
├── goal_mission_orchestrator.launch.py
├── sensors/
│   └── sick_tim_custom.launch.py

```

4.5 SSH Communication and Remote Access

Secure Shell (SSH) was used as the main modality for remote access and intersystem communication between an external computer, the Jackal onboard computer, and the external Jetson AGX Orin subsystem. This included remote system management and trigger-based coordination for the recording pipeline. (Figure 4.5)

- **SSH Access to Jackal (Wi-Fi / mDNS)**

Since the Jackal is connected to a mobile Wi-Fi network without a fixed static IP, access is established using mDNS (multicast DNS) resolution via the hostname:

```
ssh administrator@cpr-j100-0796.local
```

This avoids dependency on dynamically assigned IP addresses and allows consistent access across sessions. The hostname `cpr-j100-0796.local` is broadcast on the network and resolved automatically by the client machine. This method was preferred over static IP assignment because the robot operates in dynamic network environments (hotspots) where IP addresses may change frequently.

- **SSH Access to Jetson (Static Ethernet Network)**

In contrast, the Jetson AGX Orin is connected via a dedicated Ethernet interface (192.168.5.X subnet), which was configured with a static IP for deterministic communication. The Jetson is accessible using:

```
ssh jetson@192.168.5.1
```

This static addressing ensures reliable connectivity for low-latency command execution, which is critical for synchronising camera recording with robot actions.

SYSTEM REMOTE ACCESS & SSH COMMUNICATION ARCHITECTURE

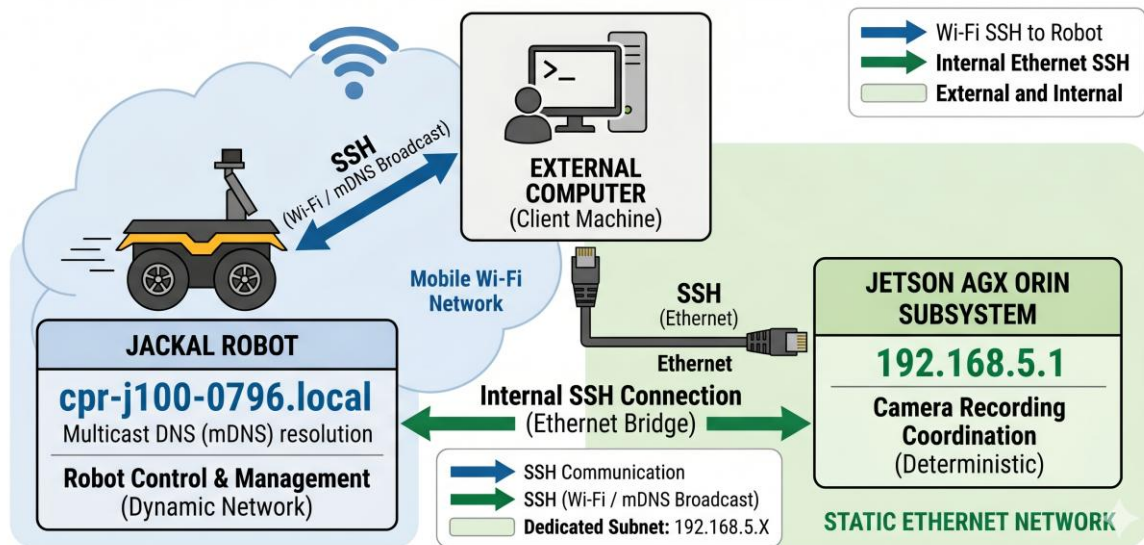


Figure 4.5: Communication method between system components

4.6 System Optimisation

Initial system testing showed that the system had serious performance limitations when running localisation and navigation at the same time. The utilisation of the CPU hits 100%, and degradation in the performance of motion is severely observed. The robot was very unstable, moving in 1-second bursts followed by several seconds of delay, indicating blocked key processes due to overload.

Further analysis showed that the localisation node (AMCL) was the main reason for this issue. During the runtime, repeated warnings of dropped messages due to overwritten timestamps were issued implying that data from the LiDAR was not being rendered in real-time. This resulted in scan messages accumulating without being processed, which led to a perpetual increase in CPU workload, inhibiting system stability.

To this, a critical optimisation was introduced at a data processing level. To this end, a vital optimisation was made at the data processing level: a mechanism was implemented to track the LiDAR scan queue size and, when the number of unprocessed messages passed a defined number (10 messages), older data in the queue was discarded. It meant that the localisation could not try to work with outdated scans and had to use fresh sensor data.

This modification significantly reduced the computational burden on the localisation process and eliminated the accumulation of stale data. As a result:

- Message dropping due to timestamp mismatch was stopped
- Localisation became stable and responsive
- CPU usage dropped from 100% to approximately 60% for the whole system, allowing other nodes to run concurrently

In addition to this primary optimisation, further tuning was applied:

- Costmap resolution (grid size) was maintained at 0.05 m, but unnecessary high-resolution configurations were avoided to limit processing overhead
- Node update frequencies (e.g., controller and costmaps) were reduced to balanced values (typically 5–10 Hz) to avoid excessive computation

4.7 Camera System Implementation

The camera subsystem was implemented as a distributed, multi-source visual acquisition system designed to capture a complete environmental view around the Jackal during experiments. The final architecture consists of eight synchronized camera streams, combining both local and network sources. The system is centred on a Jetson AGX Orin, which performs all final processing and recording. Three additional Jetson/Xavier units act as remote camera nodes, each streaming two cameras over RTSP (Real-Time Streaming Protocol). This design distributes the computational load and avoids USB bandwidth limitations on a single device.

4.7.1 System Architecture and Deployment

The camera pipeline runs inside a Docker container using NVIDIA DeepStream and GStreamer. Using a container let the programs run isolated from the host OS. Additionally, the system uses GPU acceleration for decoding, compositing, and encoding. Below, a simplified version of container configuration is shown.

```
services:
  deepstream_jetpack6:
    image: custom-deepstream:7.1-jetpack6
    runtime: nvidia
    network_mode: host
    privileged: true
    volumes:
      - ./output:/app/output
      - ./pipeline_scripts:/app/pipeline_scripts
    command: python3 2usb_n_6rtsp_to_file.py
```

Key specifications:

- `runtime: nvidia` ⇒ enables GPU acceleration (critical for 8 streams)
- `network_mode: host` ⇒ direct access to RTSP streams without port mapping
- `privileged: true` ⇒ access to `/dev/video*` USB cameras
- volume mounts ⇒ persist recordings outside the container

The process of recording starts with simple script which is launched via SSH. This script launches Docker Compose in detached mode, which allows for integration with the Jackal's mission workflow while keeping vision processing decoupled from navigation.

4.7.2 Camera Input Configuration

Investigating the need for this project, the cameras input setting is set to 1920×1080 resolution @ 30 FPS. The user can customise the input configuration at any time through changing recording parameters in relevant JSON files.

Currently Jetson AGX Orin device has two USB cameras with the address of:

```
"/dev/video0", "/dev/video2"
```

In addition to six RTSP streams from three remote units:

```
rtsp://IP:8554/cam1  
rtsp://IP:8554/cam2
```

- **USB Camera Input**

USB cameras are handled using v4l2src with enforced MJPEG input:

```
source = Gst.ElementFactory.make("v4l2src")  
caps = "image/jpeg, width=1920, height=1080, framerate=30/1"
```

Key points:

- MJPEG reduces USB bandwidth load
- Hardware decoding is offloaded to GPU (nvv4l2decoder)
- Timestamps are generated at source (do-timestamp=True)

- **RTSP Camera Input**

These inputs are processed via:

```
rtspsrc → rtph264depay → h264parse → nvv4l2decoder
```

For RTSP streams, TCP transport and configurable latency (e.g., ~400 ms) are used to improve reliability under network variations.

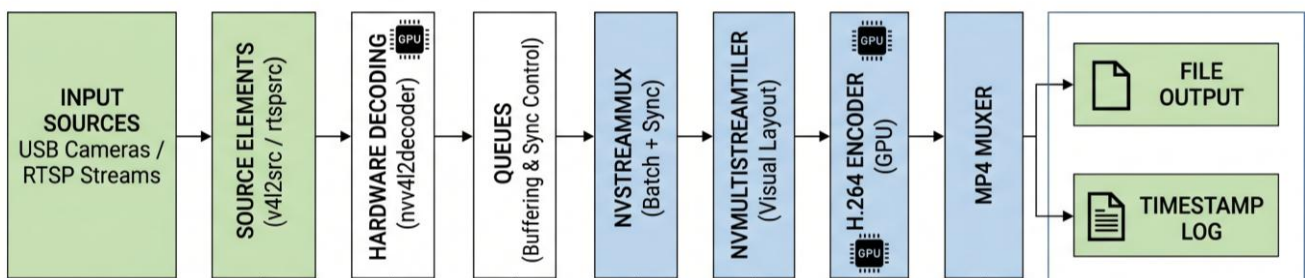


Figure 4.7.3: Overall data flow in the camera system

4.7.3 Data Flow

The camera processing pipeline is illustrated in Figure 4.7.3, showing the end-to-end data flow from multi-source input to final recorded outputs. The system accepts both USB camera inputs (`v4l2src`) and RTSP network streams (`rtspsrc`), these inputs are first decoded using hardware-accelerated decoders (`nvv4l2decoder`) to convert compressed video (H.264 or MJPEG) into raw frames. In order to buffer incoming data and regulate timing differences between the four sources, intermediate elements are used. This allows stable multi-stream processing. The decoded frames are then passed to `nvstreammux`, which performs batching and temporal synchronisation across all input streams. This is followed by the `nvmultistreamtiler`, which arranges the individual camera views into a single composite frame. The combined output is encoded using the Jetson's hardware H.264 encoder (`nvv4l2h264enc`) and multiplexed into an MP4 container. At the same time, timestamp information is extracted and logged to a separate file, which enables post-processing synchronisation between visual data and robot sensor data. This pipeline design ensures efficient GPU utilisation, synchronised multi-camera recording, and structured data output suitable for downstream analysis.

4.7.4 Multi-Camera Synchronisation

At the heart of the system, `nvstreammux` is responsible for batching all eight videos together:

```
streammux.set_property("batch-size", 8)
streammux.set_property("sync-inputs", 1)
streammux.set_property("frame-duration", 33333333) # ~30 FPS
```

Where key synchronisation parameters are:

```
"sync_settings": {
  "sync_inputs": 1,
  "max_latency_ns": 200000000,
  "batched_push_timeout_ms": 70,
  "usb_queue_max_buffers": 150,
  "rtsp_queue_max_buffers": 150
}
```

There is a logic behind this design. First, the reason for use of large buffers is to avoid frame drops during recording. Secondly, to prioritise completeness over real-time delay, higher latency tolerance is implemented, and finally, frame duration enforcement is used to have consistent temporal alignment.

4.7.5 Visual Composition

In order to have a single visual output from the eight cameras, `nvmultistreamtiler` is used. This produces a video consisting of 4 rows and 2 columns. The first row shows USB inputs and the rest demonstrate RTSP cameras.

4.7.6 Encoding, Output and Timestamp Logging

In the final stage of the pipeline, the synchronised multi-camera stream is encoded using the Jetson's hardware H.264 encoder (nvv4l2h264enc) and stored as an MP4 file. The encoder is configured for high-quality recording:

```
"encoder": {  
  "bitrate": 50000000,  
  "iframeinterval": 30,  
  "profile": "Profile_High"  
}
```

This corresponds to a 50 Mbps bitrate, with a keyframe interval of approximately 1 second (30 frames), providing a balance between video quality and efficient playback. GPU-based encoding ensures that the process has minimal impact on CPU resources.

The final video output is stored as:

```
synced_octo_output_tmstamp.mp4
```

MP4 muxing is configured with faststart, allowing the file to be played immediately after recording without additional processing.

In parallel with video recording, the system generates a timestamp log in JSON format:

```
synced_octo_output_tmstamp_timestamps.json
```

A simplified structure of this file is:

```
{  
  "recording_info": {  
    "source_camera": "usb_cam1",  
    "frame_rate_fps": 30  
  },  
  "frames": [...]  
}
```

This log is supposed to include frame-level timing information specified using the wall clock timestamp and the elapsed time, based on the reference camera stream. Using the timestamp information, during post-processing, it is possible to align the visual data with robot states, LiDAR data, and navigation-related events with precision, linking the video and ROS bags.

4.7.7 Remote Camera Units (RTSP Servers)

Each remote Jetson/Xavier runs a lightweight pipeline (2usb_to_2rtsp.py) that:

```
USB Camera → Encode → RTP → RTSP server  
  
# Example Output endpoints:  
rtsp://<ip>:8554/cam1  
rtsp://<ip>:8554/cam2
```



Figure 4.7.8-a: UNITPACKPOWER U004 52V 20AH Triangle Electric Bike Battery (Banggood, 2026)

These nodes encode the videos locally, which helps to reduce bandwidth usage and also offload processing from the main AGX Orin.

4.7.8 Camera System Power and Wiring

The camera subsystem is powered by an external Unit Pack Power (UPP) U004 48V e-bike battery, which is a Lithium-ion pack with a nominal voltage of 48 V and a maximum charge voltage of 54.6 V, generally rated at 20 Ah. The battery is designed for 750 W–1500 W high-power applications, which makes it ideal for powering the Jetson AGX Orin and the multi-camera system during prolonged data collection sessions.

Since system components operate at lower voltages, a staged DC-DC conversion architecture was put in place. Two Mean Well DDR-60L-12 converters step down the high-voltage battery output (~48–54.6 V) and regulate it into a 12 V DC rail. The two industrial DIN rail-mounted Mean Well DDR-60L-12 converters are rated at 60 W each and are connected in parallel to maximize load balancing for the system.

The 12 V rail directly powers the main compute unit (Jetson AGX Orin) and related camera hardware. Besides, a secondary DC-DC converter is used to step down from 12 V to 5 V, with specific purposes to power the network switch, which is responsible for handling RTSP camera streams entering from the distributed Jetson/Xavier units.



Figure 4.7.8-b: Mean Well DDR-60L-12 converter (WiAutomation, 2026)

Table 4.7, summarise the camera system and each component functionality. Also, Figure 4.7.8-c shows the wiring (for power) diagram of the camera system.

Table 4.7: multi-camera system summary and the function of each component in the system

Component	Function
AGX Orin	Central processing and recording
Jetson/Xavier nodes	Remote RTSP camera providers
DeepStream pipeline	Multi-stream processing
nvstreammux	Synchronisation and batching
nvmultistreamtiler	Video composition
H.264 encoder	Efficient GPU encoding
Timestamp logger	Frame-level temporal alignment
SCP transfer	Data offloading to Jackal

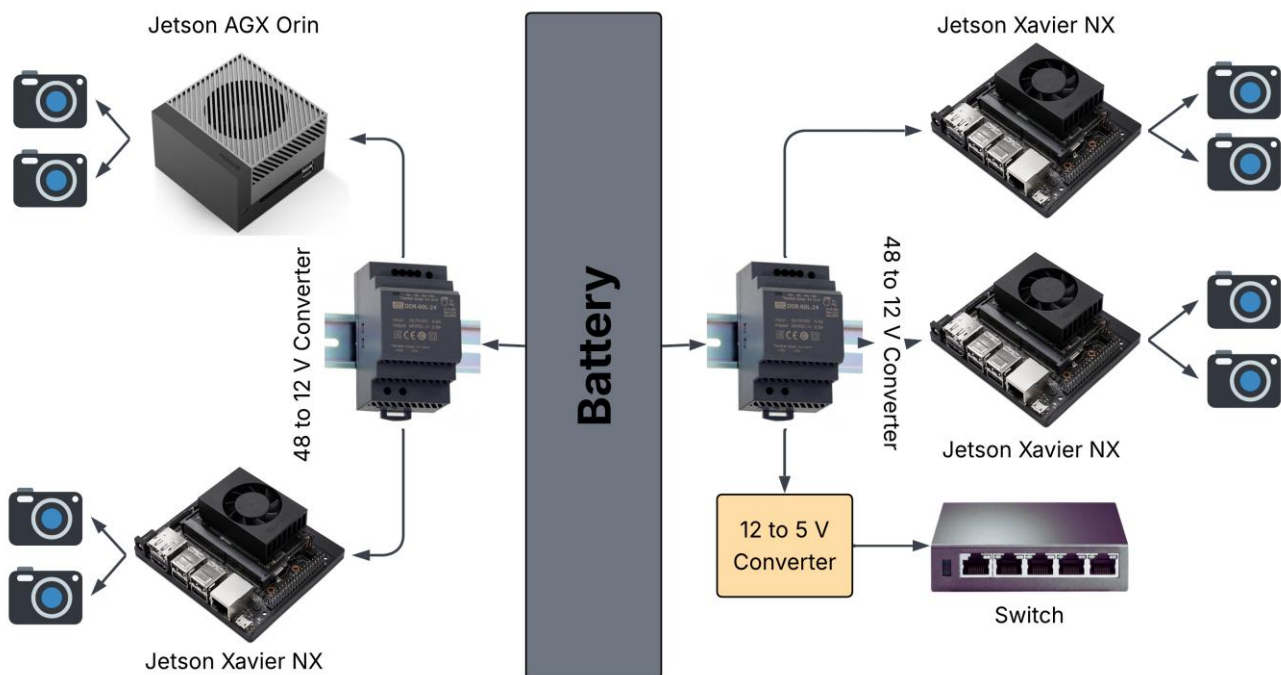


Figure 4.7.8-c: Demonstration of how to power each component of the camera system

4.8 Summary

The base environment reconstruction, workspace design, and configuration, in addition to node development and system integration, were all part of the system’s full implementation presented in this chapter. The integration of external Jetson based recording system is described with key design features such as SLAM, localization, navigation and the orchestrating pipeline.

By iterative tuning and optimisation, the finalised platform typically proved to be a good compromise with respect to performance, stability, and flexibility, supporting multiple modes of operation including autonomous navigation, teleoperation and passive recording of sensor data. Thus, the final solution after iterative tuning and optimisation provides a robust and scalable solution for mobile multi-modal data acquisition in real-world experimental scenarios.

5 System Results and Outputs

This chapter includes a discussion of the mapping results, navigation performance, multi-modal data recorded, and the overall system behaviour during experimentation. The results show that the platform is reliable in different modes and generates synchronised datasets.

5.1 Dataset Overview

The developed system was used to collect a dataset consisting of 53 recording sessions, covering multiple operation modes and environments. The dataset includes both robot sensor data (ROS bags) and synchronised multi-camera recordings.

It worth mentioning that collecting datasets is not the goal of this project, the main target was to design and implement the data collection system. According to this, the recording sessions took place in order to examine the workflow and health of the designed platform.

Table 5.1 shows the diversity of recordings. The diversity of the dataset is important, as it includes different robot behaviours, navigation conditions, and environmental contexts.

Table 5.1: Dataset summary

Category	Description
Total missions	53
Autonomous runs	34 (Nav2-based)
Teleoperation runs	13
Passive recordings	6
Environments	Indoor (41) and outdoor (12)
Navigation algorithms	Grid-based (Dijkstra), A* (configurable), DWB controller
Data types	ROS bag + multi-camera video + timestamps

5.2 Mapping Results

Mapping was performed using SLAM Toolbox, generating occupancy grid maps with a resolution of 0.05 m. The system successfully produced consistent maps across different environments, demonstrating stable LiDAR integration and TF alignment. Figure 5.2 demonstrates the result for mapping system of jackal.

The generated maps show:

- Clear obstacle boundaries
- Consistent wall alignment
- Minimal distortion over repeated passes

These results confirm that the LiDAR configuration and SLAM parameters were correctly tuned.



Figure 5.2: Aston University's main building maps; image on the left shows the map of MB-306 and the image on the right shows the map of LG-39 test room.

5.3 Localisation - Navigation Performance

Navigation package was evaluated using the Nav2 stack with different planner configurations. The system successfully understands its position in the room and execute goal-based navigation using the `/j100_0796/navigate_to_pose` action interface.

Figure 5.3-a shows a screenshot of RViz while navigating in a known map. Based on the observations, it became apparent that the navigation stack is working properly with sufficient precision since the robot followed planned trajectories with acceptable deviation.

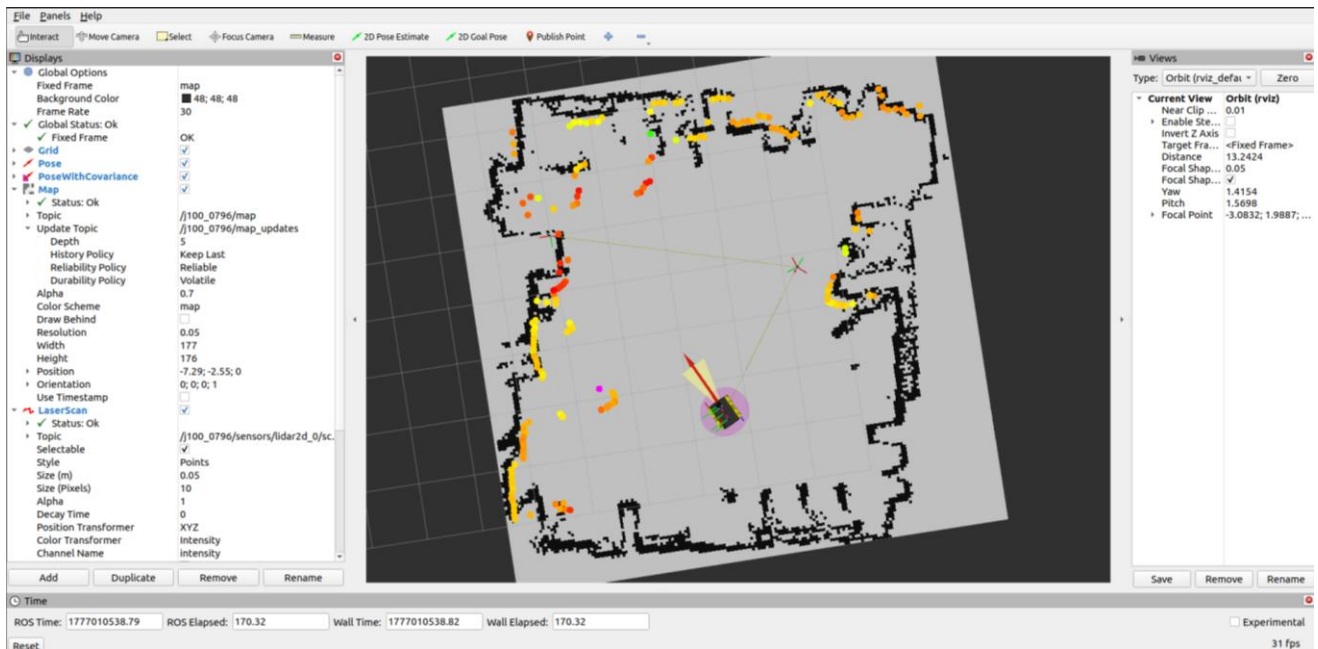


Figure 5.3-a: RViz screenshot of robot's localisation performance in a known map (MB-306)

In order to bring up navigation, three separate launch files should be initialised. The first one is LiDAR, where if the launch initiates correctly, the user will receive this message:

```
[sick_generic_caller-1] [INFO] [1777009496.346672402] [sick_scan_xd]: Setup
completed, sick_scan_xd is up and running. Pointcloud is published on topic
"cloud"
```

In the same way, the second launch file is for localisation, where the user will receive this message after a successful launch:

```
[amcl-2] [INFO] [1777009583.386989144] [j100_0796.amcl]: createLaserObject
[amcl-2] [WARN] [1777009583.394583133] [j100_0796.amcl]: AMCL cannot publish a
pose or update the transform. Please set the initial pose...
```

Finally, the third terminal is dedicated to Navigation launch file, where you receive this message after reaching to the target defined whether within terminal or RViz:

```
[controller_server-1] [INFO] [1777009845.325045156]
[j100_0796.controller_server]: Passing new path to controller.
[controller_server-1] [INFO] [1777009857.654128105]
[j100_0796.controller_server]: Reached the goal!
```

Among the points of strength of this project, the ability of the platform to perform navigation both indoor and outdoor must be highlighted. This ability roots in two factors: first, the proper chassis design of Jackal and outdoor air-filled tires enable the robot to overcome uneven surfaces and secondly, the software-wise configuration and implementation of various navigation algorithms help to reach a reliable motion. Figure 5.3-b shows the robot performing navigation tasks in various environments.

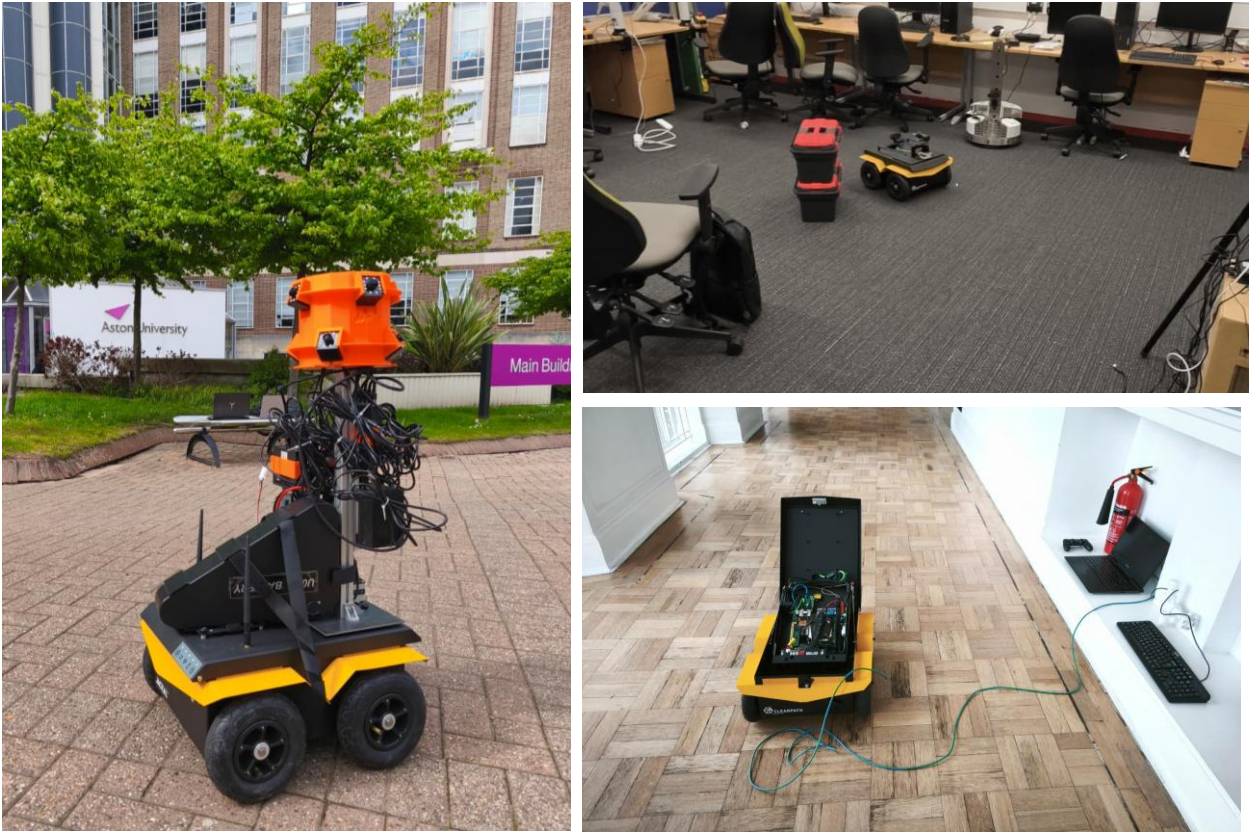


Figure 5.3-b: Jackal is capable of autonomous navigating indoor and outdoor. As these images show, this ability is tested in various scenarios.

5.4 Multi-Modal Recording Output

A key outcome of this project is the integration of robot data and visual data recording.

5.4.1 ROS Bag Data

To evaluate the real-time performance and data acquisition capability of the developed system, a rosbag recording was analysed. The dataset captures the interaction between localisation, navigation, sensing, and control subsystems over a representative operational window.

The recorded session spans approximately 4.8 seconds, during which a total of 4021 messages were generated across 75 distinct ROS 2 topics. This high message density within a short duration reflects the system's ability to sustain simultaneous multi-modal data streams without interruption.

Table 5.4.1: Frequency of the key published topics during the trial

Subsystem	Topic	Frequency (Hz)	Role
Transform (TF)	<code>/tf</code>	~96 Hz	Frame transformations
GPS	<code>/gps</code>	~43 Hz	Global positioning
IMU / Magnetometer	<code>/imu</code>	~39 Hz	Orientation estimation
Motor Feedback	<code>/motors/feedback</code>	~39 Hz	Actuator monitoring
LiDAR	<code>/scan</code>	~14 Hz	Environment sensing
Velocity Commands	<code>/cmd_vel</code>	~9 Hz	Motion control

- **Multi-Rate System Behaviour**

The system operates as a multi-rate architecture, where different subsystems publish data at varying frequencies depending on their functional role. Table 5.4.1 summarises the observed frequencies of key subsystems.

These results demonstrate a hierarchical data flow, where high-frequency streams (TF, IMU) support real-time localisation, while lower-frequency streams (LiDAR, control commands) govern planning and interaction with the environment.

- **Navigation Stack Activity**

The rosbag data confirms active participation of all major navigation components. The global planner produced a limited number of path updates, while the local planner and controller operated at higher frequencies, continuously refining motion commands.

This behaviour is consistent with the Nav2 architecture, where:

- Global planning operates at low frequency (goal-level decisions)
- Local planning reacts dynamically to obstacles
- Controllers generate continuous velocity commands

The presence of consistent `/cmd_vel` messages and corresponding motor feedback confirms that control commands are successfully propagated and executed at the hardware level.

It should be noted that the analysed dataset represents a short-duration recording (~4.8 s), and therefore frequency estimates are derived as representative values rather than long-term averages. Figure 5.4.1 shows snapshots of the recorded rosbag.

```
- topic_metadata:
  name: /j100_0796/received_global_plan
  type: nav_msgs/msg/Path
  serialization_format: cdr
  offered_qos_profiles: "- history: 1\n depth: 1\n reliability: 1\n durability: 2\n deadline:\n sec: 9223372036\n nsec: 854775807\n"
  message_count: 51
- topic_metadata:
  name: /j100_0796/platform/mcu/status/stop
  type: clearpath_platform_msgs/msg/StopStatus
  serialization_format: cdr
  offered_qos_profiles: "- history: 1\n depth: 1\n reliability: 2\n durability: 2\n deadline:\n sec: 9223372036\n nsec: 854775807\n"
  message_count: 4
- topic_metadata:
  name: /j100_0796/particle_cloud
  type: nav_msgs/msg/ParticleCloud
  serialization_format: cdr
  offered_qos_profiles: "- history: 1\n depth: 5\n reliability: 2\n durability: 2\n deadline:\n sec: 9223372036\n nsec: 854775807\n"
  message_count: 7

legion@legion:~$ ros2 bag info ros_bag/

Files:                mission_20260415_092216_812088f3_0.db3
Bag size:              49.1 MiB
Storage id:            sqlite3
Duration:              4.803631410s
Start:                 Apr 15 2026 10:22:17.148305946 (1776244937.148305946)
End:                   Apr 15 2026 10:22:21.951937356 (1776244941.951937356)
Messages:              4021
```

Figure 5.4.1: Sample of a recorded ROS bag



Figure 5.4.2: A sample of camera system output video array

5.4.2 Camera Recording Output

The camera system produced synchronised multi-view recordings using the DeepStream pipeline. Each session generates a tiled MP4 video and a timestamp JSON file. The tiled output simplifies analysis by combining all views into a single file. The final output of the recording system will be in the format of Figure 5.4.2.

5.4.3 Synchronisation Between Modalities

The synchronisation between the camera system and ROS bag data is validated by comparing the recorded timestamps from both sources. As shown in Figure 5.4.3, the ROS bag recording starts at 20:37:22 (system time on the Jackal), while the camera timestamp log shows corresponding frame timestamps around 19:37:22. The observed one-hour difference is due to a mismatch in system-wide time configuration between the Jetson AGX Orin and the Jackal computer, rather than an actual delay in recording. When this offset is taken into account, both datasets clearly align temporally, confirming that the SSH-triggered recording pipeline successfully initiates data capture simultaneously on both systems. Furthermore, the camera JSON log provides frame-level timestamps (`pts_ms`, `wall_clock`, and `elapsed_ms`), which enables precise alignment with ROS topics during post-processing. This demonstrates that the system achieves effective synchronisation between robot state data and multi-camera visual recordings despite operating across separate hardware units.

```

administrator@cpr-j100-0796:~/new_videos/2026-04-16__20-37$ cat synced_octo_output_tmstamp_timestamps.json
{
  "frame": 106,
  "pts_ms": 6046.953,
  "wall_clock": "2026-04-16T19:37:22.031",
  "elapsed_ms": 5640.679
},
{
  "frame": 107,
  "pts_ms": 6115.117,
  "wall_clock": "2026-04-16T19:37:22.063",
  "elapsed_ms": 5672.482
},
{
  "frame": 108,
  "pts_ms": 6146.943,
  "wall_clock": "2026-04-16T19:37:22.100",
  "elapsed_ms": 5709.385
},
{
  "frame": 109,
  "pts_ms": 6214.952,
  "wall_clock": "2026-04-16T19:37:22.231",
  "elapsed_ms": 5840.81
},
{
  "frame": 110,
  "pts_ms": 6246.954,
  "wall_clock": "2026-04-16T19:37:22.332",
  "elapsed_ms": 5941.278
},
{
  "frame": 111,
  "pts_ms": 6282.964,
  "wall_clock": "2026-04-16T19:37:22.395",
  "elapsed_ms": 6004.282
},
},

administrator@cpr-j100-0796:~/jackal_bags$ ros2 bag info mission_20260416_203722_909362fe/
Files:          mission_20260416_203722_909362fe_0.db3
Bag size:       84.4 MiB
Storage id:     sqlite3
Duration:       13.479466329s
Start:          Apr 16 2026 20:37:22.483660748 (1776371842.483660748)
End:            Apr 16 2026 20:37:35.963127077 (1776371855.963127077)
Messages:      12769

```

Figure 5.4.3: The ROS bag (bottom) shows system time from the Jackal, while the camera JSON log (top) shows timestamps from the Jetson. The apparent one-hour offset is due to system clock differences, while both recordings were triggered simultaneously.

5.4.4 Temporal Performance of Video Capture

Analysis of the recorded frame timestamps demonstrates that the video acquisition pipeline operates with a stable temporal resolution. Consecutive frame timestamps indicate an average inter-frame interval of approximately 33–35 ms, corresponding to an effective frame rate of ~30 FPS.

Minor variations in frame intervals were observed, with deviations typically within ± 5 ms. These fluctuations are attributed to encoding latency and system-level scheduling but remain within acceptable bounds for real-time perception systems. No frame drops or discontinuities were observed in the sampled sequence, confirming the robustness of the capture pipeline under sustained operation. Figure 5.4.4 shows a sample of the JSON file archived from a recording test. These snapshots show that between the frames 32 and 46, the recording procedure was properly continued and timestamped.

Table 5.4.4: Numerical Analysis of temporal performance of the recording system

Metric	Value
Average frame interval	~33 ms
Effective FPS	~30 FPS
Max deviation	< 5 ms
Frame continuity	100%

```

{
  "frame": 32,
  "pts_ms": 2206.805,
  "wall_clock": "2026-04-16T21:33:12.530",
  "elapsed_ms": 1876.669
},
{
  "frame": 33,
  "pts_ms": 2242.638,
  "wall_clock": "2026-04-16T21:33:12.555",
  "elapsed_ms": 1901.733
},
{
  "frame": 34,
  "pts_ms": 2306.689,
  "wall_clock": "2026-04-16T21:33:12.589",
  "elapsed_ms": 1935.417
},
{
  "frame": 35,
  "pts_ms": 2342.803,
  "wall_clock": "2026-04-16T21:33:12.686",
  "elapsed_ms": 2032.963
},
{
  "frame": 36,
  "pts_ms": 2374.625,
  "wall_clock": "2026-04-16T21:33:12.721",
  "elapsed_ms": 2068.179
},
{
  "frame": 37,
  "pts_ms": 2410.637,
  "wall_clock": "2026-04-16T21:33:12.754",
  "elapsed_ms": 2101.302
},
{
  "frame": 38,
  "pts_ms": 2442.661,
  "wall_clock": "2026-04-16T21:33:12.823",
  "elapsed_ms": 2169.873
},
{
  "frame": 39,
  "pts_ms": 2474.653,
  "wall_clock": "2026-04-16T21:33:12.857",
  "elapsed_ms": 2204.406
},
{
  "frame": 40,
  "pts_ms": 2610.629,
  "wall_clock": "2026-04-16T21:33:12.890",
  "elapsed_ms": 2236.456
},
{
  "frame": 41,
  "pts_ms": 2710.636,
  "wall_clock": "2026-04-16T21:33:12.920",
  "elapsed_ms": 2267.014
},
{
  "frame": 42,
  "pts_ms": 2742.627,
  "wall_clock": "2026-04-16T21:33:12.956",
  "elapsed_ms": 2303.003
},
{
  "frame": 43,
  "pts_ms": 2778.655,
  "wall_clock": "2026-04-16T21:33:12.990",
  "elapsed_ms": 2336.821
},
{
  "frame": 44,
  "pts_ms": 2846.638,
  "wall_clock": "2026-04-16T21:33:13.124",
  "elapsed_ms": 2470.695
},
{
  "frame": 45,
  "pts_ms": 2910.634,
  "wall_clock": "2026-04-16T21:33:13.232",
  "elapsed_ms": 2578.955
},
{
  "frame": 46,
  "pts_ms": 2946.695,
  "wall_clock": "2026-04-16T21:33:13.259",
  "elapsed_ms": 2606.395
},

```

Figure 5.4.4: Showcase of a sample of the JSON file archived from a recording test.

The system records both presentation timestamps (PTS) and wall-clock timestamps for each frame, enabling precise temporal alignment between sensing modalities. Analysis of the dataset shows a consistent offset between PTS time and wall-clock time, primarily due to system-level clock configuration differences.

Despite this offset, the relative temporal consistency between consecutive frames is preserved, ensuring reliable synchronisation across data streams. The elapsed time progression closely matches the PTS increments, indicating that the recording pipeline maintains internal timing coherence.

This confirms that the system is suitable for post-processing alignment, where absolute timestamps can be corrected while preserving accurate inter-frame temporal relationships.

5.5 System Performance

System performance was evaluated by monitoring CPU utilisation during simultaneous execution of localisation, navigation, and sensor processing nodes. Figure 5.5 presents a comparison of system load before and after optimisation using the `htop` tool.

Prior to optimisation, CPU usage was consistently near 90–100% across all cores, as shown in the first screenshot. The system load average exceeded 14.0, indicating severe computational overload. At this stage, critical nodes such as `amcl`, `planner_server`, `controller_server`, and `bt_navigator` were consuming a significant portion of CPU resources. This resulted in unstable robot behaviour, with delayed processing of LiDAR data and intermittent execution of navigation commands.

After implementing optimisation strategies—primarily the LiDAR queue management mechanism and parameter tuning—the system load was significantly reduced. As shown in the second screenshot, CPU utilisation decreased to approximately 55–65% per core, with a load average around 6.6. The computational demand is now more evenly distributed across nodes, and no single process dominates CPU usage.

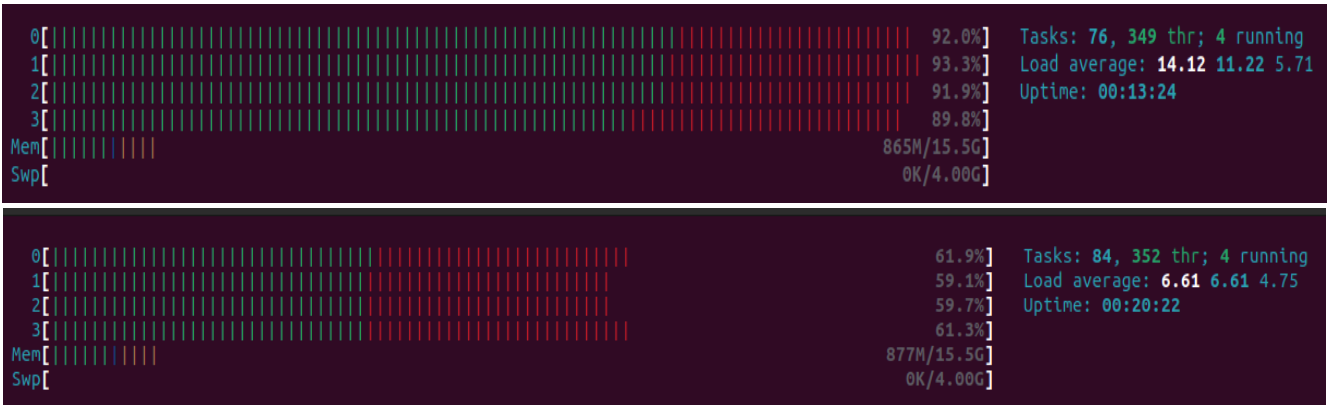


Figure 5.5: Comparison of system performance using htop. Before optimisation (top), CPU usage is near saturation, causing instability. After optimisation (bottom), CPU load is significantly reduced, resulting in smooth and reliable operation.

As a result of this improvement:

- Navigation execution became smooth and continuous
- Sensor data processing stabilised without backlog
- Message dropping due to timestamp delays was reduced
- All Nav2 and localisation nodes operated within real-time constraints

These results demonstrate that the applied optimisation techniques successfully transformed the system from an overloaded and unstable state into a reliable and real-time capable platform suitable for continuous data collection.

5.6 Summary

This chapter presented the outputs generated by the system, including maps, navigation behaviour, and multi-modal recordings. The results demonstrate that the platform is capable of:

- Reliable autonomous and manual operation
- Consistent multi-camera data acquisition
- Accurate synchronisation between robot and visual data
- Stable performance under real-world conditions

Overall, the system successfully achieves its objective of providing a robust data collection platform for mobile robotics experiments, supporting both structured and unstructured recording scenarios.

6 Discussion

This chapter assesses the developed system from an engineering and research perspective. This chapter analyses the developed system from an engineering and research standpoint and evaluates the strength of the implemented architecture, discusses major technical challenges during development, identifies limitations of the system, and presents future work directions.

6.1 System Strength

The implemented platform has been observed to possess strong qualities regarding system design, integration, and real-world practicality.

The main strength of the system is in its modular architecture. Separating navigation (Nav2), the orchestration nodes, and the camera subsystem means that no component depends on another. This design allows for massive decoupling of the system so that failure in one of the subsystems (e.g., recording, or networking) will not influence the core navigation of the robot. Also, this modular structure eases debugging and testing, as well as extending the system.

Other notable strengths include the multi-modal data acquisition feature. The recordings include:

- LiDAR and robot's states data (ROS bag)
- Multi-view camera data (8 camera DeepStream pipeline)
- Timestamp logs for synchronization

This combination yields a rich dataset that captures both environmental geometry and visual context, which can be useful for social navigation and behaviour analysis purposes. In addition, the flexibility of the system's operation mode is shown in the support for the following operational modes:

- Autonomous Navigation
- Teleoperation recording
- Passive data acquisition

This flexibility means that the platform can include a variety of scenarios, such as controlled experiments, real-world interactions, and edge cases that are hard to simulate with purely autonomous methods.

Finally, in terms of engineering strengths, distributed camera processing (Jetson AGX Orin + remote RTSP nodes) is significant. This system is achieved without overloading the critical onboard navigation processes by offloading any visual processing from the robot's onboard computer, while still producing high-quality multi-camera recording.

6.2 Engineering Challenges

There are several non-trivial technical challenges involved in developing the system, including real-time performance, synchronisation, and system integration.

One of the most paramount challenges was the problem of CPU overload during simultaneous localisation and navigation. At first, the system hit almost 100% CPU usage, which significantly deteriorate its performance. The robot motion was delayed, control signals were sent at non-uniform intervals, and certain messages from the sensors never reached their destination. The timestamp mismatches meant that the localisation node (AMCL) was attempting to process ever-growing queues of LiDAR data, saturating computational resources.

This issue was solved by the introduction of a LiDAR queue management mechanism, with a thrown away older unprocessed scan messages from the queue exceeding a predefined threshold. Therefore, localisation relied only on recent data, which dramatically reduced the computational load, and real-time behaviour was restored.

The second major challenge was to setup LiDAR sensor since this sensor is not supported from Clearpath Robotics. Configuring YAML file, defining the sensor in `robot.yaml`, and programming launch file for this sensor were challenging. However, finally the sensor is alive and working in coordination with other stacks, especially Nav2 and localisation.

Another challenge was multi-camera synchronisation on distributed hardware. Each Jetson device, which formed the camera system, had its own system clock. Therefore, discrepancies in timestamps between the robot's data and the video recording were observed. This was mitigated through:

- Simultaneous recording starts with SSH
- Frame level timestamps
- Post-processing alignment based on timestamp metadata

Although there was a slight time offset left by the difference between the system clocks, the two streams of data were effectively synchronised for analysis purposes.

Network configuration and reliability issues posed another challenge. There are multiple network interfaces used in the system:

- Wi-Fi (outside access to Jackal)
- Ethernet bridge (internal communication with sensors and Jetson)
- RTSP streaming between camera nodes

Ensuring that communication across these networks remained stable necessitated configuring the networks' IP addressing, routing, and latency management for the RTSP streams.

6.3 System Limitations

There are, however, several limitations of the system that can be taken into account in further its development.

First, it is essential to make it clear that the goal of this work was to create a solid data acquisition platform rather than to obtain a vast dataset. Though the system's primary output is still raw data recordings, this multi-modal data is currently in an unrefined state. However, in the system architecture, the necessary JSON-data format for working with such data is already defined, and further work will be done to convert recordings into this format so that the produced data can be used by machine learning directly.

In terms of computational performance, despite the major optimisation achieved, the system still operates at near its processing limits when put under full load conditions. CPU resource demands are heavily tied to the localisation, navigation, and sensor-processing functions running concurrently. This need is especially true if the additional sensors or more complex algorithms are introduced.

From a perception view, the system currently utilizes a 2D LiDAR sensor limiting the brain's understanding of the environment to a plane only. This means that the perceived data is limited to a plane or 2D projection, which is enough for navigation purposes but not for fully understanding the environment, especially if it involves complex 3D structures or actions. A 3D LiDAR sensor or a depth sensing solution would be preferable in this case.

Another limitation pertains to the mechanical structure of the camera system since during recording sessions the vibration and harsh movement of the Jackal's chassis is partially transferred to the camera ring; this calls for more structural design, including installing vibration damping or isolation devices to harness more stable visual data.

Overall, while the system satisfies its flexibility and functionality as a data acquisition platform, its limitations suggest critical areas to incorporate improvements on in respect to computational performance, sensing capacity, and mechanical design.

6.4 Comparison with Existing Approaches

The designed system has a few advantages over existing robotics datasets and platforms.

A lot of publicly available datasets are made in controlled settings or use sensors that are fixed in a same location. This system, on the other hand, lets you collect real-world data on the go, recording both robot movement and human activity in changing settings.

Combining visual data from multiple cameras with robot sensor data also makes for a better representation than datasets with only one type of data. This is especially important for things like social navigation, where both visual and spatial cues are important.

The current system, on the other hand, is more focused on the infrastructure for collecting data than on the size or benchmarking of datasets. Its main benefit is that it makes data collection flexible and repeatable, not that it makes a fully labelled dataset.

6.5 Future Work

A key direction is scaling the dataset, both in terms of the number of recording sessions and the diversity of environments. This would make the dataset more useful for research and machine learning purposes. Another important milestone is the addition of data annotation tools, which let you label recorded data for tasks like finding objects, studying human behaviour, or learning how to navigate.

From a perception perspective, the system could be enhanced by adding 3D LiDAR or depth cameras, providing richer environmental information.

Finally, the data that was already collected could be used to make and test learning-based navigation models, which would connect data collection to smart decision-making.

7 Conclusion

This project presented the design and implementation of a modular, multi-modal data acquisition platform based on the Clearpath Jackal robot. The primary objective was to develop a system capable of reliably collecting synchronised robot sensor data and multi-camera visual data under real-world operating conditions.

The system was successfully reconstructed from a non-functional initial state and redeveloped using Ubuntu 22.04 and ROS 2 Humble, establishing a stable and modern robotics framework. A structured ROS 2 workspace was implemented, enabling clear separation between configuration, navigation, and orchestration components. The integration of SLAM Toolbox, AMCL localisation, and the Nav2 stack allowed the robot to perform mapping and autonomous navigation reliably.

A key contribution of this work is the development of a custom orchestration layer, which coordinates navigation and data recording processes. Through this mechanism, the system supports multiple operational modes, including autonomous navigation, teleoperation-based recording, and passive data collection. This flexibility enables the platform to capture a wide range of real-world scenarios.

In parallel, a distributed multi-camera system was implemented using Jetson-based hardware and NVIDIA DeepStream. By combining USB and RTSP camera streams into a synchronised pipeline, the system produces high-quality multi-view recordings alongside structured timestamp logs. The integration of SSH-based triggering ensures that camera recording and ROS data collection are initiated simultaneously, enabling effective synchronisation between modalities.

Significant effort was devoted to system optimisation, particularly in addressing CPU overload during localisation and navigation. The introduction of LiDAR queue management and parameter tuning reduced computational load and enabled stable real-time operation. As a result, the system is capable of executing navigation tasks while continuously recording multi-modal data without instability.

Overall, the developed platform demonstrates that it is possible to achieve reliable, synchronised, and flexible data acquisition in a mobile robotics context using a modular and distributed architecture. The system provides a solid foundation for future work, including structured data processing, enhanced perception capabilities, and learning-based navigation approaches.

In conclusion, this project successfully delivers a scalable and extensible data collection platform, supporting further research in robotics, perception, and autonomous systems.

References

- 1 Chen, Z., Guo, Y., Chu, Z., Luo, M., Shen, Y., Sun, M., Hu, J., Xie, S., Yang, K., Shi, P. and Gu, Z., 2025. Socialnav: Training human-inspired foundation model for socially-aware embodied navigation. *arXiv preprint arXiv:2511.21135*.
- 2 Nahavandi, S., Alizadehsani, R., Nahavandi, D., Mohamed, S., Mohajer, N., Rokonzaman, M. and Hossain, I., 2025. A comprehensive review on autonomous navigation. *ACM Computing Surveys*, 57(9), pp.1-67.
- 3 Xiao, X., Liu, B., Warnell, G. and Stone, P., 2022. Motion planning and control for mobile robot navigation using machine learning: a survey. *Autonomous Robots*, 46(5), pp.569-597.
- 4 Kruse, T., Pandey, A.K., Alami, R. and Kirsch, A., 2013. Human-aware robot navigation: A survey. *Robotics and Autonomous Systems*, 61(12), pp.1726-1743.
- 5 Kaggle. "Dynamic Indoor Robot Navigation Dataset", Kaggle, 2026. [Online]. Available at: <https://www.kaggle.com/datasets/ziya07/dynamic-indoor-robot-navigation-dataset> (accessed 22 April 2026)
- 6 Francis, A., Pérez-d'Arpino, C., Li, C., Xia, F., Alahi, A., Alami, R., Bera, A., Biswas, A., Biswas, J., Chandra, R. and Chiang, H.T.L., 2025. Principles and guidelines for evaluating social robot navigation algorithms. *ACM Transactions on Human-Robot Interaction*, 14(2), pp.1-65.
- 7 Hall, E.T. and Hall, E.T., 1966. *The hidden dimension* (Vol. 609). Anchor.
- 8 Pellegrini, S., Ess, A., Schindler, K. and Van Gool, L., 2009, September. You'll never walk alone: Modeling social behavior for multi-target tracking. In *2009 IEEE 12th international conference on computer vision* (pp. 261-268). IEEE.
- 9 Bachiller-Burgos, P., Bernardet, U., Calderita, L.V., Chhetri, P., Francis, A., Hirose, N., Pérez, N., Shah, D., Singamaneni, P.T., Xiao, X. and Manso, L.J., 2025. Towards Data-Driven Metrics for Social Robot Navigation Benchmarking. *arXiv preprint arXiv:2509.01251*.
- 10 Hart, P.E., Nilsson, N.J. and Raphael, B., 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2), pp.100-107.
- 11 Macenski, S., Martín, F., White, R. and Clavero, J.G., 2020, October. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2718-2725). IEEE.
- 12 Maruyama, Y., Kato, S. and Azumi, T., 2016, October. Exploring the performance of ROS2. In *Proceedings of the 13th international conference on embedded software* (pp. 1-10).
- 13 Fox, D., Burgard, W. and Thrun, S., 2002. The dynamic window approach to collision avoidance. *IEEE*

- robotics & automation magazine*, 4(1), pp.23-33.
- 14 Keirsey, D., Koch, E., McKisson, J., Meystel, A. and Mitchell, J., 1984, March. Algorithm of navigation for a mobile robot. In *Proceedings. 1984 IEEE International Conference on Robotics and Automation* (Vol. 1, pp. 574-583). IEEE.
 - 15 Quang, H.D., Tran, T.L., Manh, T.N., Manh, C.N., Nhu, T.N. and Duy, N.B., 2022. Design a nonlinear MPC controller for autonomous mobile robot navigation system based on ROS. *International Journal of Mechanical Engineering and Robotics Research*, 11(6).
 - 16 Deitke, M., Han, W., Herrasti, A., Kembhavi, A., Kolve, E., Mottaghi, R., Salvador, J., Schwenk, D., VanderBilt, E., Wallingford, M. and Weihs, L., 2020. Robothor: An open simulation-to-real embodied ai platform. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 3164-3174).
 - 17 Martín-Martín, R., Rezatofighi, H., Shenoi, A., Patel, M., Gwak, J., Dass, N., Federman, A., Goebel, P. and Savarese, S., 2019. Jrdb: A dataset and benchmark for visual perception for navigation in human environments. *arXiv preprint arXiv:1910.11792*.
 - 18 Karnan, H., Nair, A., Xiao, X., Warnell, G., Pirk, S., Toshev, A., Hart, J., Biswas, J. and Stone, P., 2022. Socially compliant navigation dataset (scand): A large-scale dataset of demonstrations for social navigation. *IEEE Robotics and Automation Letters*, 7(4), pp.11807-11814.
 - 19 Hirose, N., Shah, D., Sridhar, A. and Levine, S., 2023. Sacson: Scalable autonomous control for social navigation. *IEEE Robotics and Automation Letters*, 9(1), pp.49-56.
 - 20 Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R. and Ng, A.Y., 2009, May. ROS: an open-source Robot Operating System. In *ICRA workshop on open source software* (Vol. 3, No. 3.2, p. 5).
 - 21 Macenski, S., Martín, F., White, R. and Clavero, J.G., 2020, October. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2718-2725). IEEE.
 - 22 Clearpath Robotics Inc. “Jackal UGV - Small Weatherproof Robot”, Clearpath Robotics Inc, 2026. [Online]. Available at: https://docs.clearpathrobotics.com/docs_robots/outdoor_robots/jackal/user_manual_jackal/ (accessed 22 April 2026)
 - 23 Clearpath Robotics Inc. “Jackal UGV - Small Weatherproof Robot”, Clearpath Robotics Inc, 2026. [Online]. Available at: https://docs.clearpathrobotics.com/docs_robots/outdoor_robots/jackal/maintenance_jackal/ (accessed 22 April 2026)
 - 24 Banggood Global leading online shop. “UNITPACKPOWER U004 52V 20AH 2500mAh Triangle

Electric Bike Battery with 30A BMS Protection Board and 3A Charger for Max 1100W Motor Kit”, Banggood Global leading online shop, 2026. [Online]. Available at: https://fr.banggood.com/EU-Direct-UNITPACKPOWER-U004-52V-20AH-Ebike-TriangleBattery-2500mAh-Battery-with-30A-BMS-Protection-Board-3A-Charger-for-Max-1100W-Motor-Kit-p-2017895.html?cur_warehouse=CZ (accessed 22 April 2026)

- 25 WiAutomation. “DDR-60L-12 Mean Well”, WiAutomation, 2026. [Online]. Available at: https://gb.wiautomation.com/mean-well/power-supply/DDR60L12?utm_source=google&utm_medium=cpc&utm_campaign=UK_pmax_full&gad_source=1&gad_campaignid=17418022258&gbraid=0AAAAAC2XDKCi6RHP0Z-RBQm9Q_W0IMxyF&gclid=CjwKCAjwhqfPBhBWEiwAZo196jN1rwR6vGkg07KPgFSsVXLgZWwSeoVM86j7A8GTcYG75rQIxNjUkxoCW2YQAvD_BwE (accessed 22 April 2026)

Appendix

The complete implementation of the system, including source code, configuration files, and deployment scripts, is available in the following GitHub repository:

GitHub Repository:

https://github.com/Legionnnnn/jackal_clearpath_ws.git

The repository includes:

- ROS 2 workspace structure and custom packages
- Launch files and configuration (YAML) files for Nav2 and sensors
- Data acquisition and orchestration scripts
- Sample datasets, including rosbag recordings and video outputs

Additional resources, such as demonstration videos and extended datasets, are also provided to support reproducibility and further research.